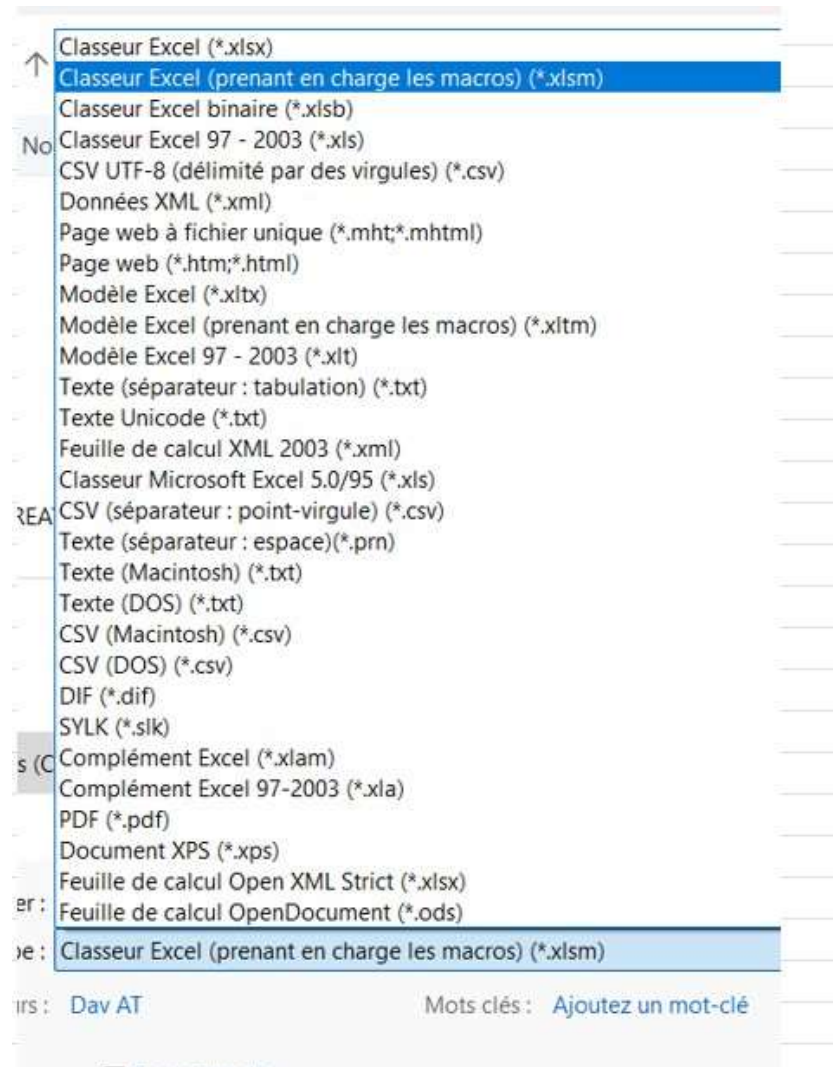


VBA EXCEL

Introduction aux généralités de Visual Basic dans Excel

Le format xlsx obligatoire



Accès à l'onglet développeur

Cocher la case développeur

Options Excel

Générales

Formules

Données

Vérification

Enregistrement

Langue

Accessibilité

Options avancées

Personnaliser le ruban

Barre d'outils Accès rapide

Compléments

Centre de gestion de la confidentialité



Personnaliser le ruban.

Choisir les commandes dans les catégories suivantes :

Commandes courantes

- Actualiser tout
- Ajouter ou supprimer des filtres
- Annuler
- Aperçu et impression
- Augmenter la taille de police
- Bordures
- Calculer maintenant
- Centrer
- Collage spécial
- Coller
- Coller
- Copier
- Couleur de police
- Couleur de remplissage
- Couper
- Définir
- Enregistrer
- Enregistrer sous
- Envoyer par courrier
- Exposant
- Figurer les volets
- Format de cellule
- Formes
- Fusionner et centrer

Ajouter >>

<< Supprimer

Personnaliser le ruban :

Onglets principaux

Onglets principaux

- > ☒ Suppression de l'arrière-plan
- > ☒ Accueil
- > ☐ FORMULAIRES (Personnalisé)
- > ☒ Insertion
- > ☒ Dessin
- > ☒ Mise en page
- > ☒ Formules
- > ☒ Données
- > ☒ Révision
- > ☒ Affichage
 - > Modes d'affichage
 - > Afficher
 - > Zoom
 - > Fenêtre
 - > Macros
- > ☐ Développeur
- > ☒ Compléments
- > ☒ Aide

Nouvel onglet

Nouveau groupe

Renommer...

Personnalisations :

Réinitialiser

Importer/Exporter

OK

Annuler

Création d'onglet, de groupe et de commande

Options Excel

- Générales
- Formules
- Données
- Vérification
- Enregistrement
- Langue
- Accessibilité
- Options avancées
- Personnaliser le ruban**
- Barre d'outils Accès rapide
- Compléments
- Centre de gestion de la confidentialité

Personnaliser le ruban.

Choisir les commandes dans les catégories suivantes :

Commandes courantes

- Actualiser tout
- Ajouter ou supprimer des filtres
- Annuler
- Aperçu et impression
- Augmenter la taille de police
- Bordures
- Calculer maintenant
- Centrer
- Collage spécial
- Coller
- Coller
- Copier
- Couleur de police
- Couleur de remplissage
- Couper
- Définir
- Enregistrer
- Enregistrer sous
- Envoyer par courrier
- Exposant
- Figurer les volets
- Format de cellule
- Formes
- Fusionner et centrer

Ajouter >>

<< Supprimer

Personnaliser le ruban :

Onglets principaux

Onglets principaux

- > ☒ Suppression de l'arrière-plan
- > ☒ Accueil
- > ☐ FORMULAIRES (Personnalisé)
- > ☒ Insertion
- > ☒ Dessin
- > ☒ Mise en page
- > ☒ Formules
- > ☒ Données
- > ☒ Révision
- > ☒ Affichage
 - > Modes d'affichage
 - > Afficher
 - > Zoom
 - > Fenêtre
 - > Macros
- > ☐ Développeur
- > ☒ Compléments
- > ☒ Aide

Nouvel onglet

Nouveau groupe

Renommer...

Personnalisations :

Réinitialiser

Importer/Exporter

OK

Annuler

Personnalisation du ruban

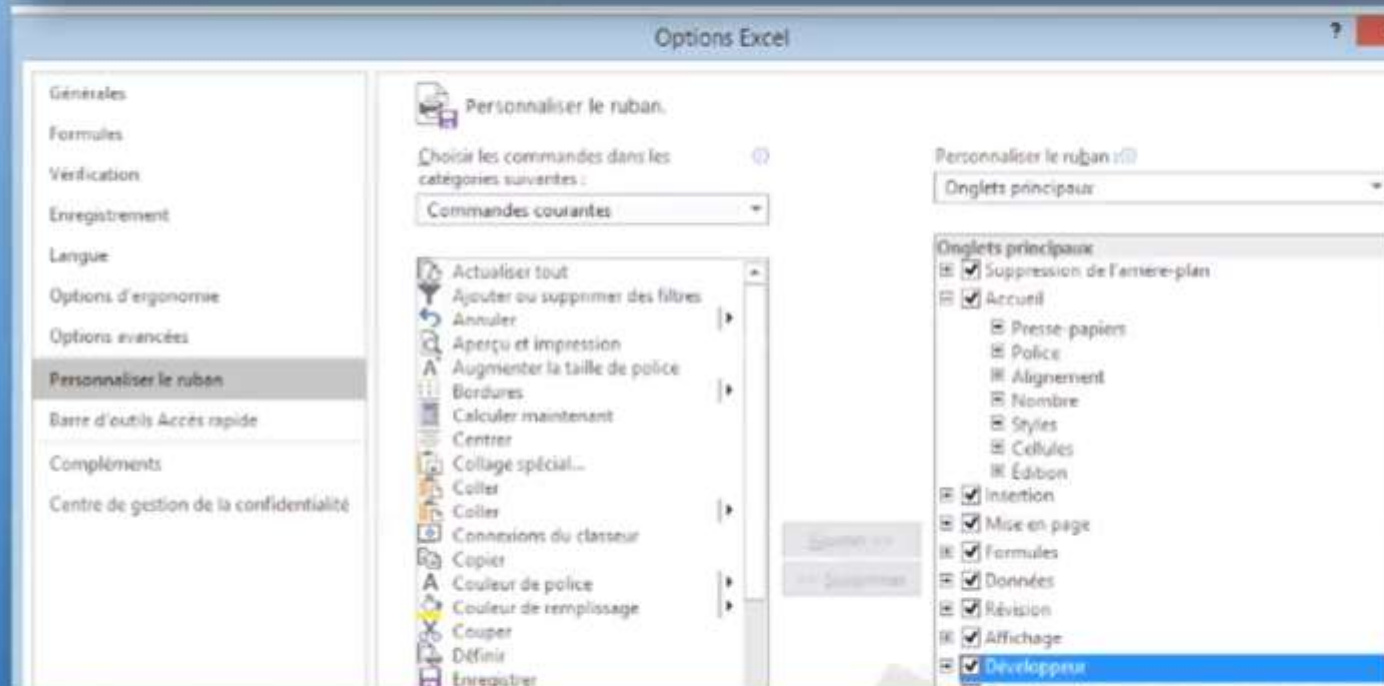
les menus vba



Personnalisation du ruban

Personnalisation du ruban

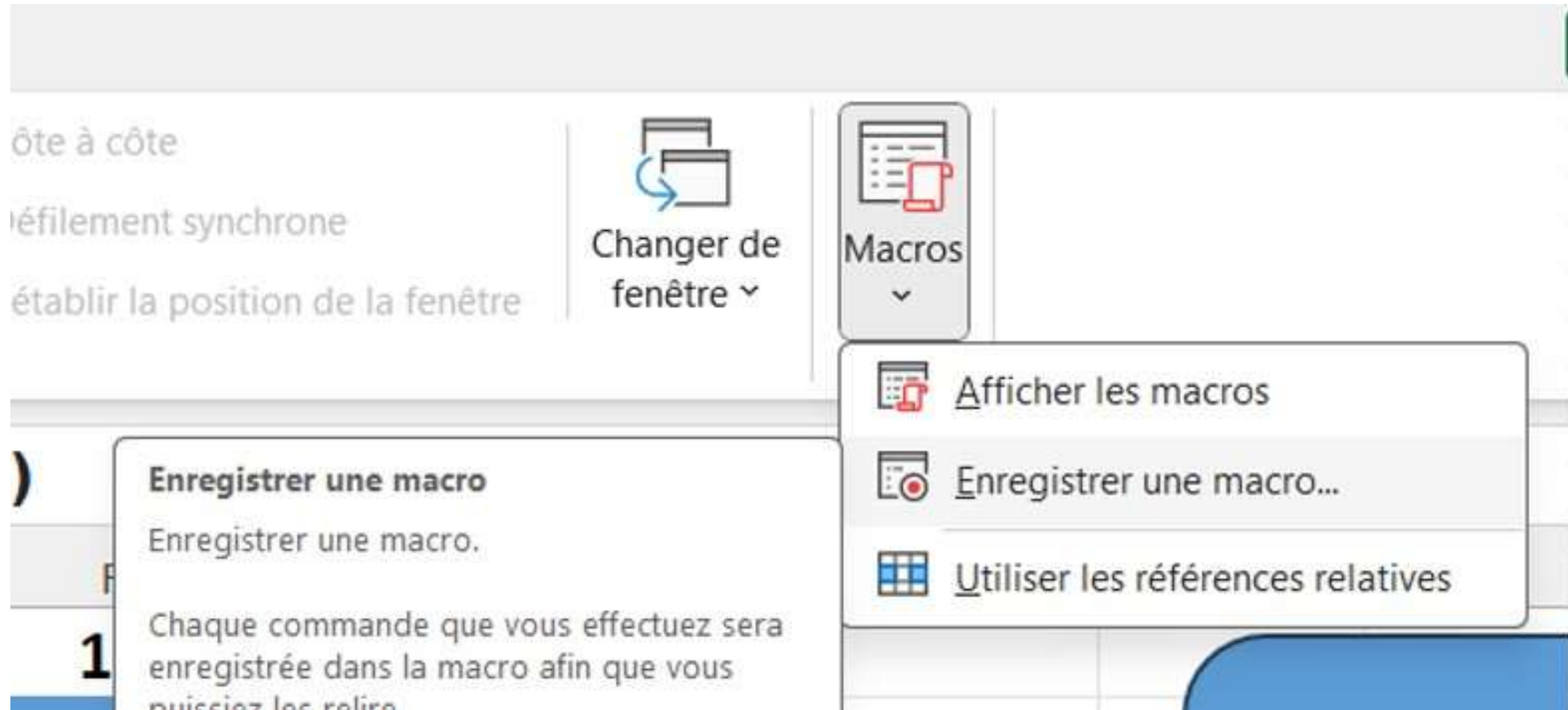
Cliquez sur **Fichier** > **Options** > **Personnaliser le Ruban** puis cochez **Développeur** :



Un nouvel onglet sera ajouté

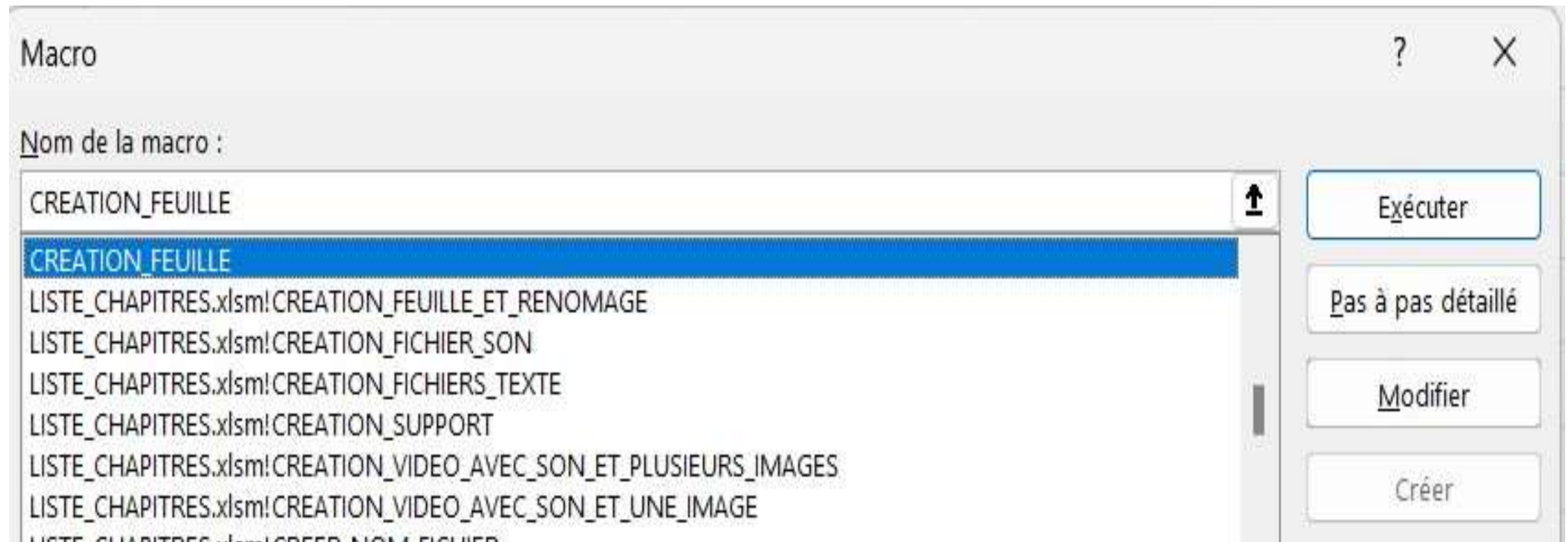
Utilisation de l'enregistreur de macros

Enregistrement d'une macro



Création et édition de macros

Edition d'une macro



Création et édition de macros

La modification des macros enregistrées

```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select  
    Application.CutCopyMode = False  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    Range("D3").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    Range("E3").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    Range("F3").Select  
    ActiveCell.FormulaR1C1 = "MONDE"  
    Range("F4").Select
```

```
End Sub
```

```
Sub SAISIE_TEXTE_BIS()
```

```
    ActiveCell.Select  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "MONDE"
```

Types d'enregistrement

ABOLU OU RELATIF

```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select  
    Application.CutCopyMode = False  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    Range("D3").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    Range("E3").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    Range("F3").Select  
    ActiveCell.FormulaR1C1 = "MONDE"  
    Range("F4").Select
```

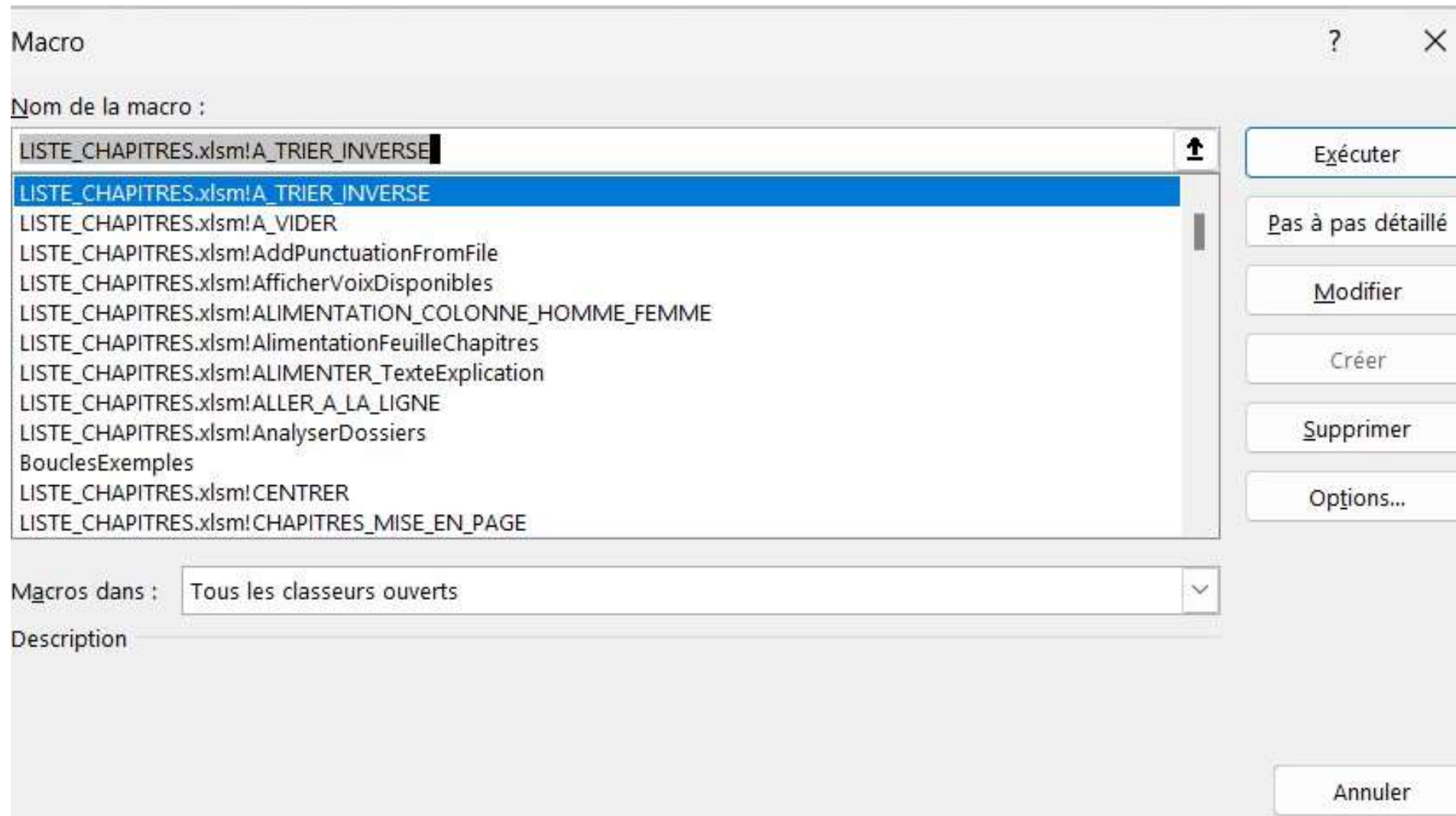
```
End Sub
```

```
Sub SAISIE_TEXTE_BIS()
```

```
    ActiveCell.Select  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "MONDE"
```

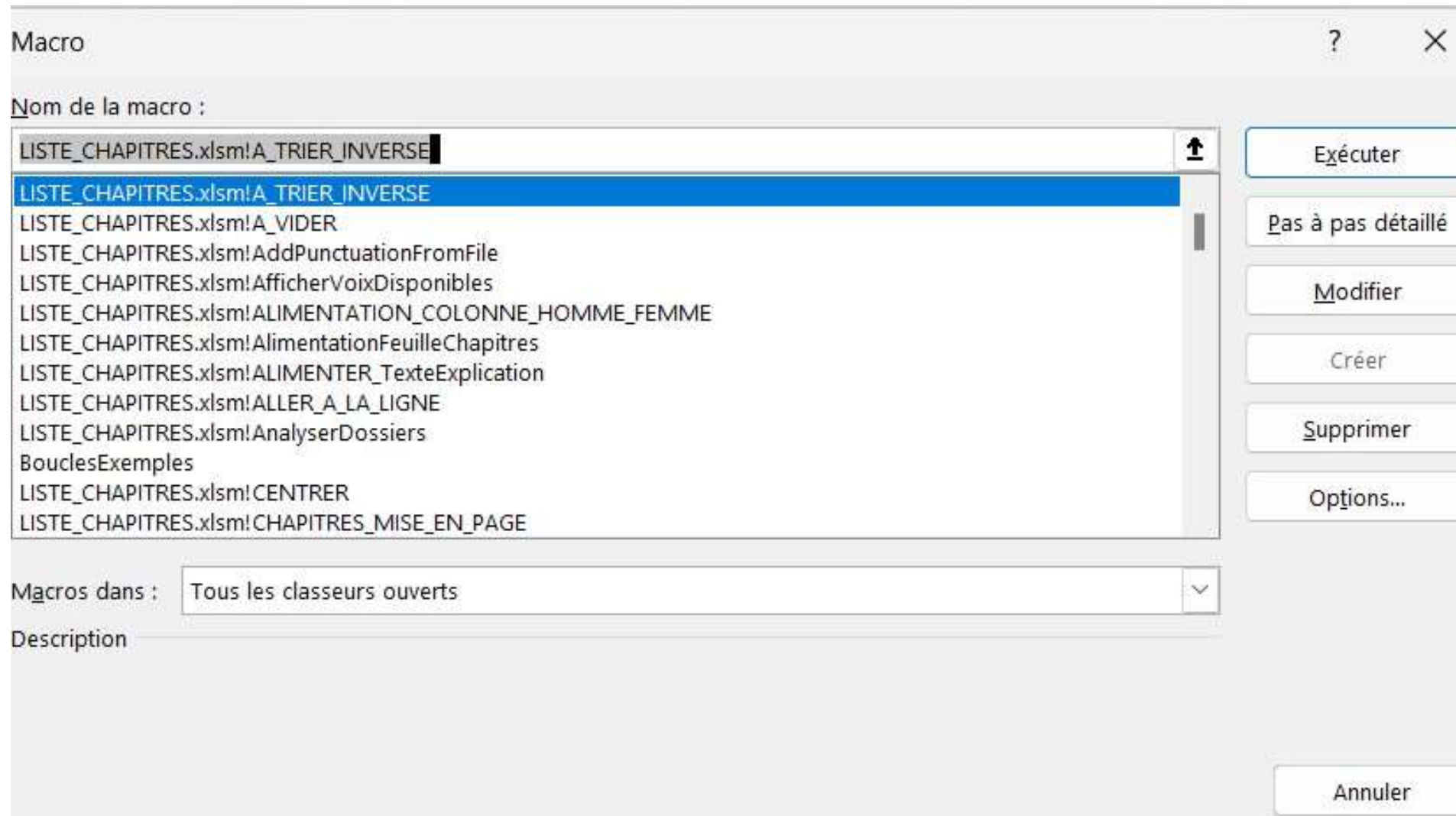

Appel des macros par la liste des macros

Affichage de la liste de macros



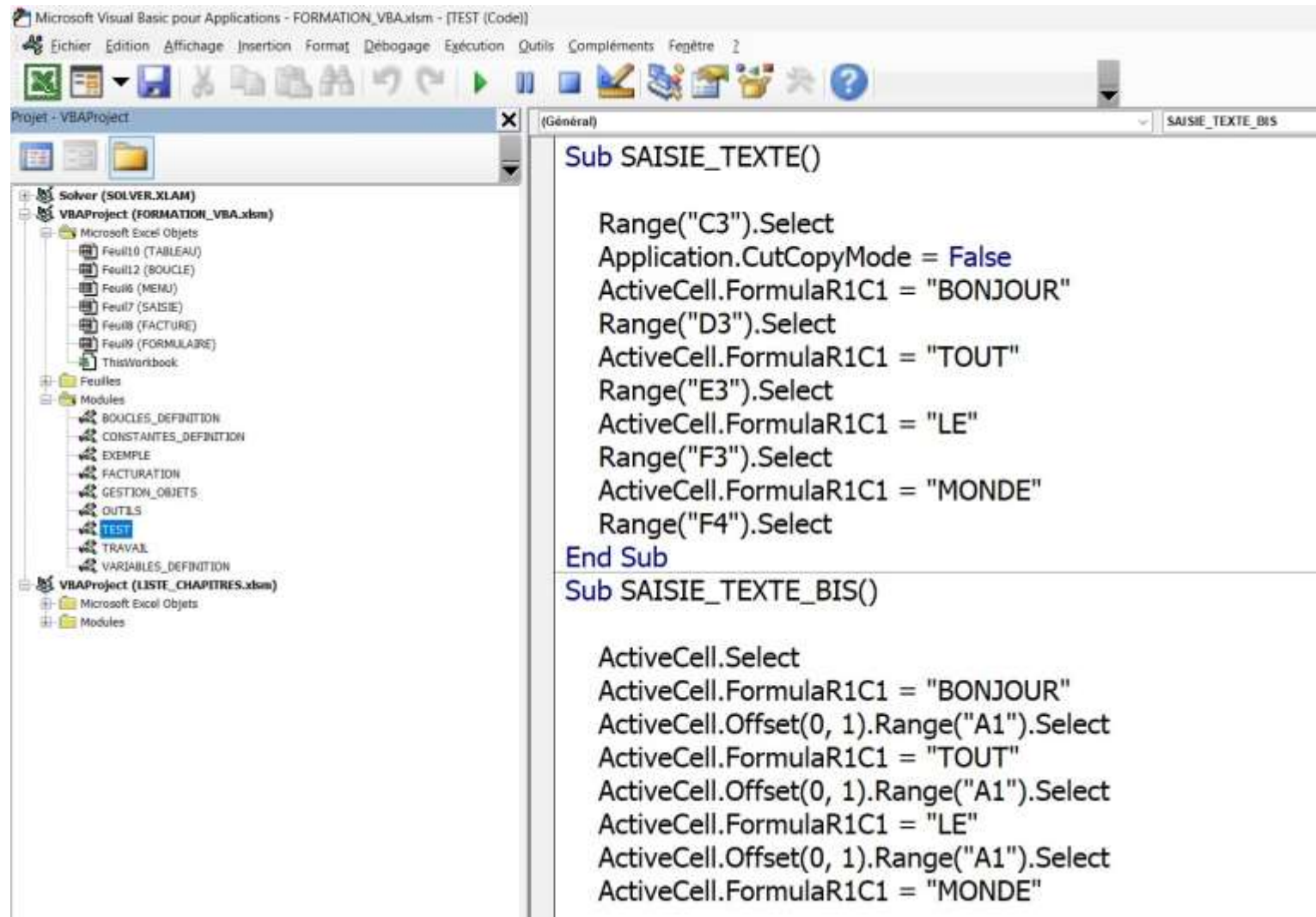
Appel des macros par la liste des macros

Executer une macro a partir de l'affichage des macros



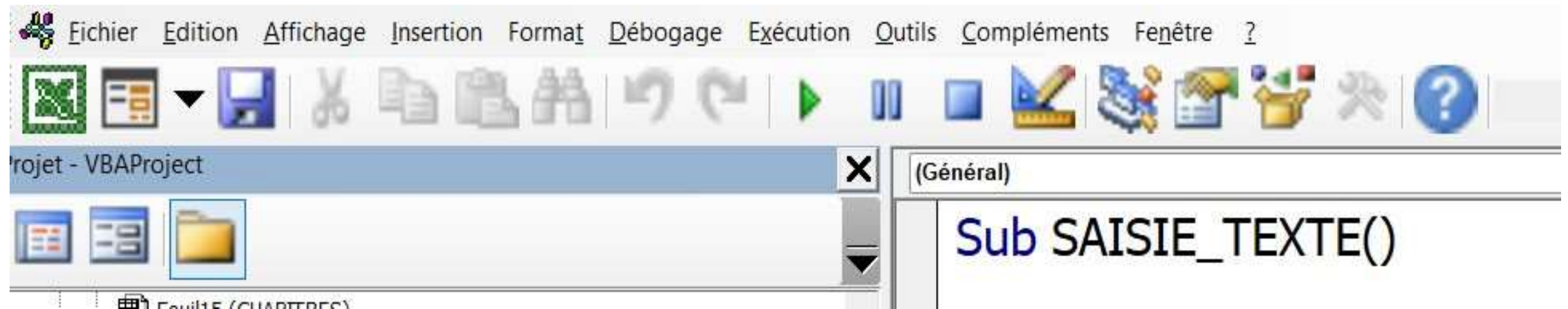
Exploration de l'environnement VBA

Exploration de l'environnement vba



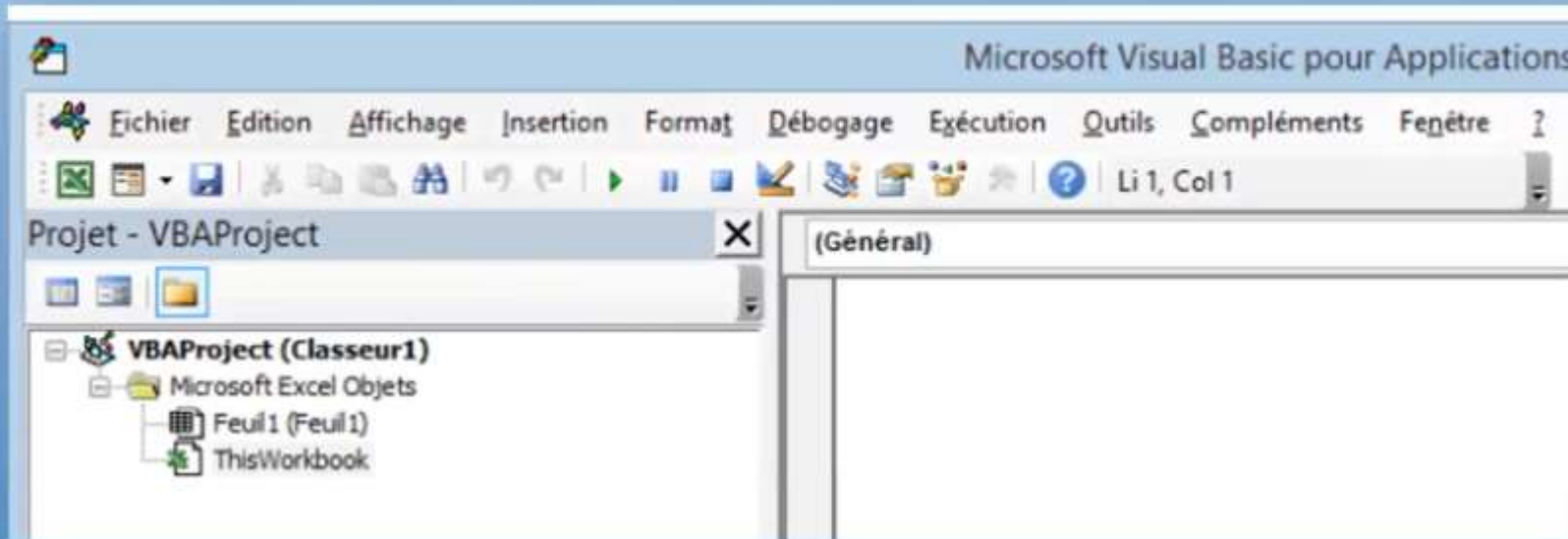
Découverte des fenêtres, menus et outils

Decouverte des fenetres menus et outils



Utilisation de l'explorateur de projets

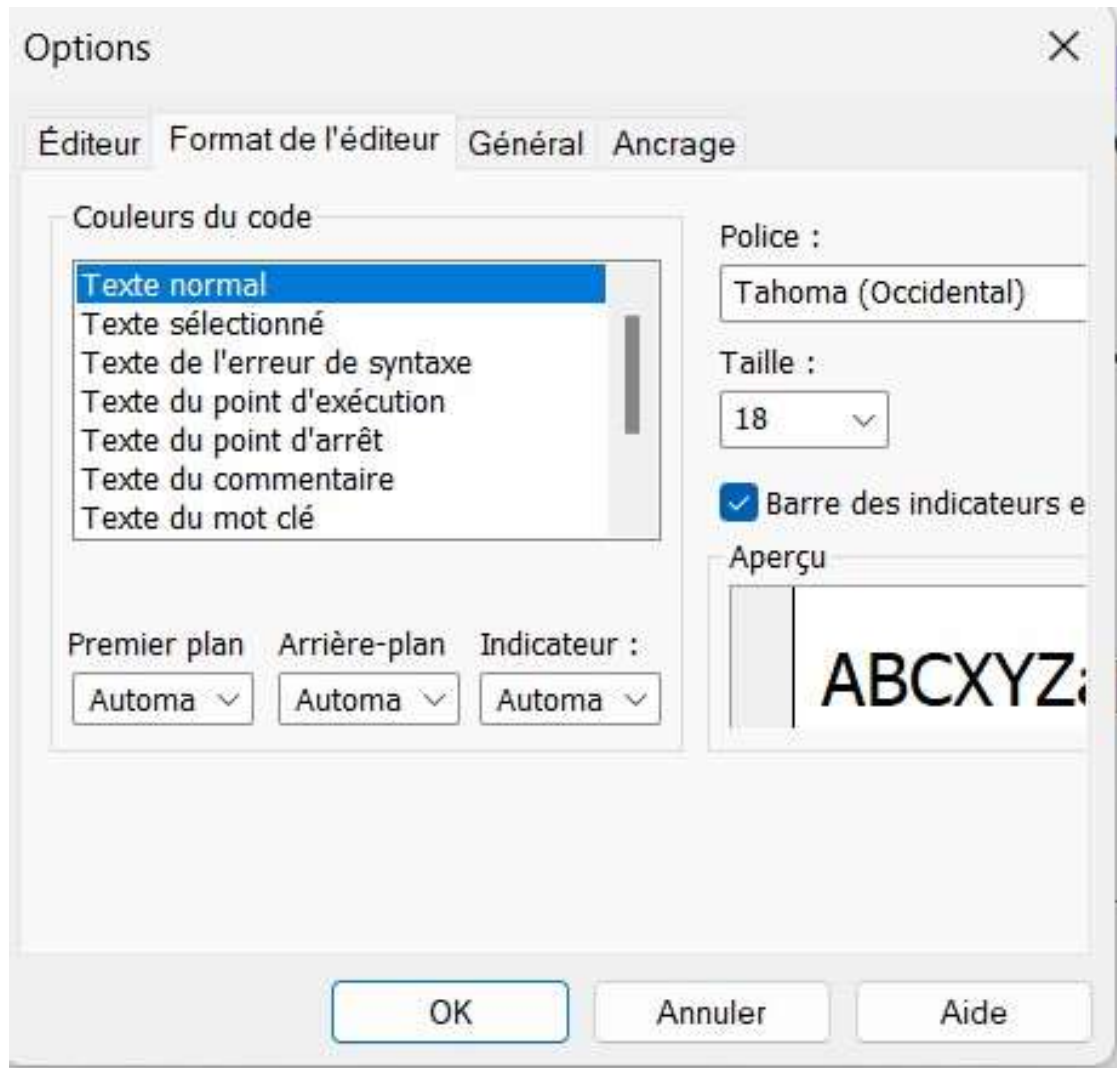
l'explorateur de projets



Nous y reviendrons, retenez simplement le raccourci **Alt + F11** pour le moment

Personnalisation des options VBA

Personnalisation des options VBA



Approfondissement du code VBA

Definition ActiveCell

```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select  
    Application.CutCopyMode = False  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    Range("D3").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    Range("E3").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    Range("F3").Select  
    ActiveCell.FormulaR1C1 = "MONDE"  
    Range("F4").Select
```

```
End Sub
```

```
Sub SAISIE_TEXTE_BIS()
```

```
    ActiveCell.Select  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "MONDE"
```

Approfondissement du code VBA

Definition ActiveSheet

```
Sub RENOMMAGE_FEUILLE()
```

```
    ActiveSheet.Name = "TEST"
```

```
End Sub
```


Approfondissement du code VBA

Definition de la plage de cellules

```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select  
    Application.CutCopyMode = False  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    Range("D3").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    Range("E3").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    Range("F3").Select  
    ActiveCell.FormulaR1C1 = "MONDE"  
    Range("F4").Select
```

```
End Sub
```

Approfondissement du code VBA

Definition de Nothing

' Libérer les objets

Set v_stream = Nothing

Set v_fso = Nothing

Approfondissement du code VBA

Definition de Option Explicit

(Général)

Option Explicit

```
Sub CREATION_SUPPORT()  
    Dim v_pptApp As Object  
    Dim v_pptPres As Object  
    Dim v_pptSlide As Object  
    Dim v_feuille As Worksheet  
    Dim v_derniereLigne As Long
```

Lecture et simplification du code VBA

Lecture et simplification du code VBA

```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select  
    Application.CutCopyMode = False  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    Range("D3").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    Range("E3").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    Range("F3").Select  
    ActiveCell.FormulaR1C1 = "MONDE"  
    Range("F4").Select
```

```
End Sub
```

```
Sub SAISIE_TEXTE_BIS()
```

```
    ActiveCell.Select  
    ActiveCell.FormulaR1C1 = "BONJOUR"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "TOUT"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "LE"  
    ActiveCell.Offset(0, 1).Range("A1").Select  
    ActiveCell.FormulaR1C1 = "MONDE"
```

Complétion d'une macro existante

Comment reperer une cellule

```
' Parcourir la colonne F de la 3ème ligne à la dernière ligne remplie  
For v_ligneCourante = 3 To v_derniereLigne  
    v_feuille.Cells(v_ligneCourante, 6).Value = v_valeurs(v_Index)  
    v_Index = (v_Index + 1) Mod 4 ' Passer à la valeur suivante de manière circulaire  
Next v_ligneCourante
```


Complétion d'une macro existante

Complétion d'une macro existante

```
v_FEUILLE_FORMULAIRE.Select
```

```
v_CLIENT = v_FEUILLE_FORMULAIRE.Cells(4, 4).Value
```

```
v_NUMERO = v_FEUILLE_FORMULAIRE.Cells(5, 4).Value
```

```
v_DATE = v_FEUILLE_FORMULAIRE.Cells(6, 4).Value
```

```
v_MONTANT_HT = v_FEUILLE_FORMULAIRE.Cells(7, 4).Value
```

```
v_TVA = RECUP_CALCUL_TVA(v_MONTANT_HT)
```

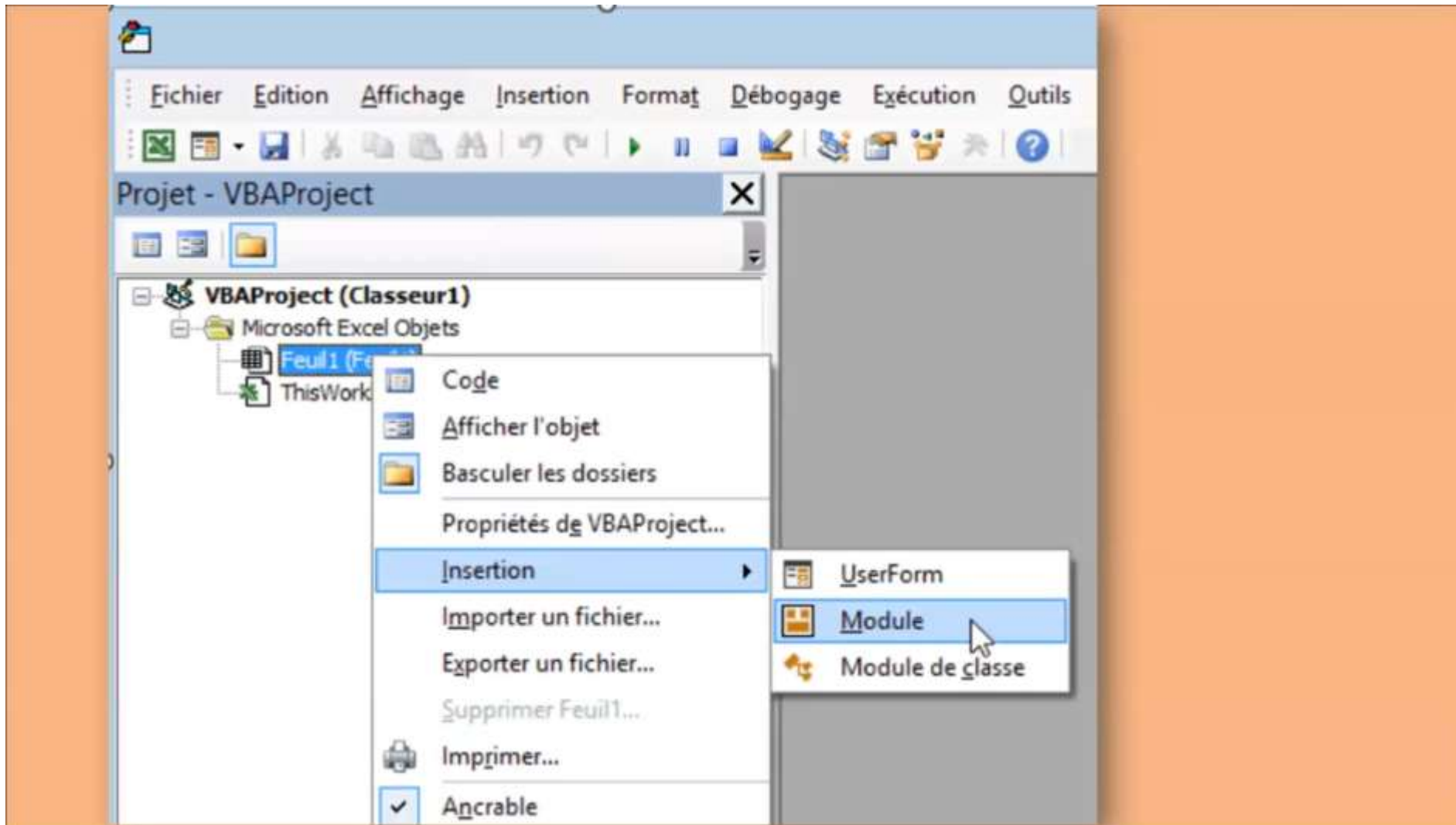
```
v_TTC = RECUP_CALCUL_TTC(v_MONTANT_HT, v_TVA)
```

```
v_FEUILLE_FORMULAIRE.Cells(9, 4).Value = v_TVA
```

```
v_FEUILLE_FORMULAIRE.Cells(10, 4).Value = v_TTC
```


Structure d'un module VBA

les modules



Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

Sub et Function

LES FONCTIONS

La principale différence entre Sub et Function est qu'une fonction retourne une valeur.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

```
Function carre(nombre As Double)

    carre = nombre ^ 2 'La fonction "carre" retourne la valeur de "carre"

End Function

Sub exemple()

    Dim resultat As Double

    resultat = carre(9.876) 'La variable resultat reçoit la valeur retournée par la fonction

    MsgBox resultat 'Affichage du résultat (ici, le carré de 9.876)

End Sub
```

Sub et Function

Vous pouvez remarquer que les arguments d'une fonction sont ajoutés entre `()` contrairement aux procédures où elles sont superflues.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

Abs	Renvoie la valeur absolue d'un nombre.
CInt / CLng	Convertit une valeur numérique en nombre entier en arrondissant à l'entier le plus proche.
CSng / CDbI	Convertit une valeur numérique en nombre à virgule.
Int / Fix	Renvoie la partie entière d'un nombre.
RGB	Renvoie un nombre entier correspondant à une couleur au format RGB.
Rnd	Renvoie un nombre aléatoire inférieur à 1 et supérieur ou égal à 0.
Round	Arrondit un nombre en fonction du nombre de décimales spécifié.
Val	Renvoie les nombres contenus dans une chaîne de caractères.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

FONCTIONS EXCEL

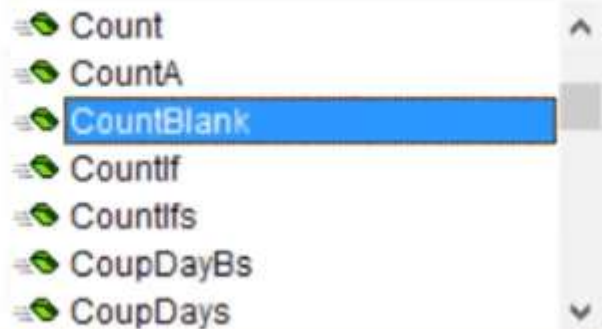
Il est également possible d'utiliser les fonctions Excel dans le code VBA.

Après avoir entré `WorksheetFunction` suivi d'un `.`, la liste des fonctions apparaît :

```
Sub exemple()
```

```
    test = WorksheetFunction.
```

```
End Sub
```



- Count
- CountA
- CountBlank
- CountIf
- CountIfs
- CoupDayBs
- CoupDays

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Les fonctions sont toutes en anglais

Fonction en français	Fonction en anglais	Description de la fonction
NB	COUNT	Détermine les nombres compris dans la liste des arguments.
NB.SI	COUNTIF	Compte le nombre de cellules qui répondent à un critère donné dans une plage.
NB.SI.ENS	COUNTIFS	Compte le nombre de cellules à l'intérieur d'une plage qui répondent à plusieurs critères.
NB.VIDE	COUNTBLANK	Compte le nombre de cellules vides dans une plage.
NBVAL	COUNTA	Détermine le nombre de valeurs comprises dans la liste des arguments.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

La fonction choisie pour cet exemple est donc **NB.VIDE**.

```
Sub exemple()  
    test = WorksheetFunction.CountBlank(|  
                                           CountBlank(Arg1 As Range) As Double  
End Sub
```

L'exemple suivant affiche le nombre de cellules vides de la plage **A1:D8** dans la boîte de dialogue :

```
Sub exemple()  
  
    MsgBox WorksheetFunction.CountBlank(Range("A1:D8"))  
  
End Sub
```

Déclarations, procédures, fonctions et commentaires

Comment réaliser une fonction

FONCTIONS VBA

créer une fonction

personnalisée

Pour cet exemple, nous allons créer une fonction SI personnalisée utilisable sur une feuille de calcul comme n'importe quelle autre fonction d'Excel.

Pour afficher la valeur de la colonne B si la cellule de la colonne C contient OUI , nous pouvons utiliser la formule `=SI(C2="OUI";B2;0)` :

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

L'objectif ici est de créer une fonction capable de faire cela `=SI(C2 a un fond vert;B2;0)` que nous écrirons comme ceci : `=SI_VERT(C2;B2)` :

	A	B	C	D	E
1	Factures	Montant	Payé	Montant déjà payé (si vert)	
2	XLP-838893	100.00 €			
3	XLP-838894	200.00 €			
4	XLP-838895	300.00 €			
5	XLP-838896	400.00 €			
6	XLP-838897	500.00 €			
7	XLP-838898	600.00 €			
8	XLP-838899	700.00 €			
9	XLP-838900	800.00 €			
10	XLP-838901	900.00 €			
11	XLP-838902	1'000.00 €			
12	XLP-838903	1'100.00 €			
13			Total payé :	0.00 €	

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

DÉVELOPPER LA FONCTION PERSONNALISÉE

Commencez par créer votre fonction :

```
Function SI_VERT(celluleCouleur As Range, celluleValeur As Range)  
  
End Function
```

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Les arguments :

- `celluleCouleur As Range` : la cellule dont il faut tester la couleur
- `celluleValeur As Range` : la cellule contenant le montant

Dans le cas présent, si le test est FAUX, la valeur sera 0 à chaque fois, c'est pour cela qu'un 3^e argument n'a pas été ajouté.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Pour vérifier si la couleur est correcte, vous pouvez utiliser une cellule contenant la bonne couleur comme point de comparaison :

```
Function SI_VERT(celluleCouleur As Range, celluleValeur As Range)  
  
    couleur = Sheets("Feuil1").Range("K1").Interior.color  
  
End Function
```


Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Mais pour éviter de dépendre d'une cellule, nous allons utiliser ici directement le numéro de la couleur qui nous intéresse :

```
Function SI_VERT(celluleCouleur As Range, celluleValeur As Range)  
  
    couleur = 5296274 'Vert  
  
End Function
```

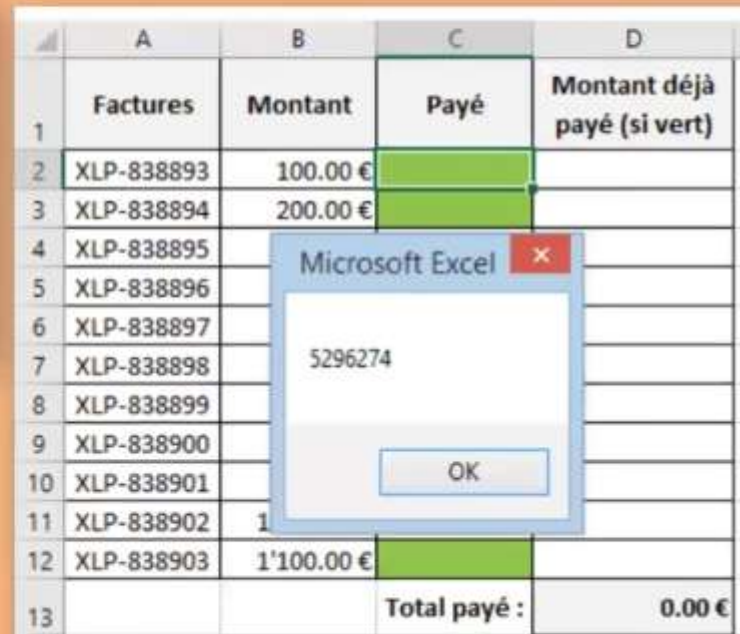
Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Pour connaître le numéro de couleur de fond d'une cellule, sélectionnez la cellule et exécutez cette macro

```
Sub test()  
  
    MsgBox ActiveCell.Interior.color  
  
End Sub
```



The screenshot shows an Excel spreadsheet with the following data:

	A	B	C	D
1	Factures	Montant	Payé	Montant déjà payé (si vert)
2	XLP-838893	100.00 €		
3	XLP-838894	200.00 €		
4	XLP-838895			
5	XLP-838896			
6	XLP-838897			
7	XLP-838898			
8	XLP-838899			
9	XLP-838900			
10	XLP-838901			
11	XLP-838902	1		
12	XLP-838903	1'100.00 €		
13			Total payé :	0.00 €

A Microsoft Excel dialog box is open, displaying the value 5296274, which is the interior color index of the selected cell (C2). The dialog box has an 'OK' button and a red 'X' in the top right corner.

Déclarations, procédures, fonctions et commentaires

Comment realiser une fonction

FONCTIONS VBA

Il ne reste plus qu'à tester la couleur avec un **If** :

```
Function SI_VERT(celluleCouleur As Range, celluleValeur As Range)

    Application.Volatile

    If celluleCouleur.Interior.color = 5296274 Then 'Si VRAI
        SI_VERT = celluleValeur
    Else 'Si FAUX
        SI_VERT = 0
    End If

End Function
```

Déclarations, procédures, fonctions et commentaires

Comment réaliser une fonction

FONCTIONS VBA

```
Sub actualiser()  
  
    Application.Calculate  
  
End Sub
```

D2					
	A	B	C	D	E
1	Factures	Montant	Payé	Montant déjà payé (si vert)	
2	XLP-838893	100.00 €		100.00 €	
3	XLP-838894	200.00 €		200.00 €	
4	XLP-838895	300.00 €		0.00 €	
5	XLP-838896	400.00 €		400.00 €	
6	XLP-838897	500.00 €		0.00 €	
7	XLP-838898	600.00 €		600.00 €	
8	XLP-838899	700.00 €		700.00 €	
9	XLP-838900	800.00 €		0.00 €	
10	XLP-838901	900.00 €		900.00 €	
11	XLP-838902	1'000.00 €		0.00 €	
12	XLP-838903	1'100.00 €		1'100.00 €	
13			Total payé :	4'000.00 €	Actualiser

les procédures et fonctions

PUBLIC - PRIVATE

Pour le moment, toutes les procédures créées sont de type `Public`, elles sont accessibles depuis tous les modules.

```
Sub exemple()
```

'Est identique à :

```
Public Sub exemple()
```

Déclarations, procédures, fonctions et commentaires

Comment realiser une procedure

Pour rendre une procédure inaccessible hors du module, ajoutez `Private` :

```
Private Sub exemple()
```


Déclarations, procédures, fonctions et commentaires

Comment realiser une procedure

LANCER UNE PROCÉDURE DEPUIS UNE PROCÉDURE

```
Private Sub avertissement()  
  
    MsgBox "Attention !!!"  
  
End Sub  
  
Sub exemple()  
  
    If Range("A1") = "" Then  
        avertissement ' <= exécute la procédure "avertissement"  
    End If  
  
End Sub
```

Déclarations, procédures, fonctions et commentaires

Difference entre une procedure et une fonction

```
Function RECUP_CALCUL_TVA(v_MONTANT_HT As Double) As Double
    Dim v_TVA As Double
    v_TVA = v_MONTANT_HT * 20 / 100
    RECUP_CALCUL_TVA = v_TVA
End Function
```

```
Function RECUP_CALCUL_TTC(v_MONTANT_HT As Double, v_TVA As Double) As Double
    Dim v_TTC As Double
    v_TTC = v_MONTANT_HT + v_TVA
    RECUP_CALCUL_TTC = v_TTC
End Function
```

```
Sub INSERTION_LIGNE_3()

    Rows("3:3").Select
    Selection.Insert Shift:=xlDown, CopyOrigin:=xlFormatFromLeftOrAbove
End Sub
```

```
Sub INSERTION_LIGNE_16()

    Rows("16:16").Select
    Selection.Insert Shift:=xlDown, CopyOrigin:=xlFormatFromLeftOrAbove
End Sub
```

Déclarations, procédures, fonctions et commentaires

Les commentaires

```
'  
  
' Macro1 Macro  
  
'
```

Déclarations, procédures, fonctions et commentaires

Remplacement du code par des procédures et fonctions

```
v_TVA = RECUP_CALCUL_TVA(v_MONTANT_HT)  
v_TTC = RECUP_CALCUL_TTC(v_MONTANT_HT, v_TVA)
```

```
Function RECUP_CALCUL_TVA(v_MONTANT_HT As Double) As Double  
    Dim v_TVA As Double  
    v_TVA = v_MONTANT_HT * 20 / 100  
    RECUP_CALCUL_TVA = v_TVA  
End Function
```

```
Function RECUP_CALCUL_TTC(v_MONTANT_HT As Double, v_TVA As Double) As Double  
    Dim v_TTC As Double  
    v_TTC = v_MONTANT_HT + v_TVA  
    RECUP_CALCUL_TTC = v_TTC
```

Organisation d'un module VBA

Le besoin de fabriquer des outils unitaires

```
Sub COPIE()  
    Selection.Copy  
End Sub
```

```
Sub COLLAGE()  
    ActiveSheet.Paste  
End Sub
```

Organisation d'un module VBA

Le besoin de maintenir son code

```
Sub VALIDATION()
```

```
    Call INSERTION_LIGNE_11
```

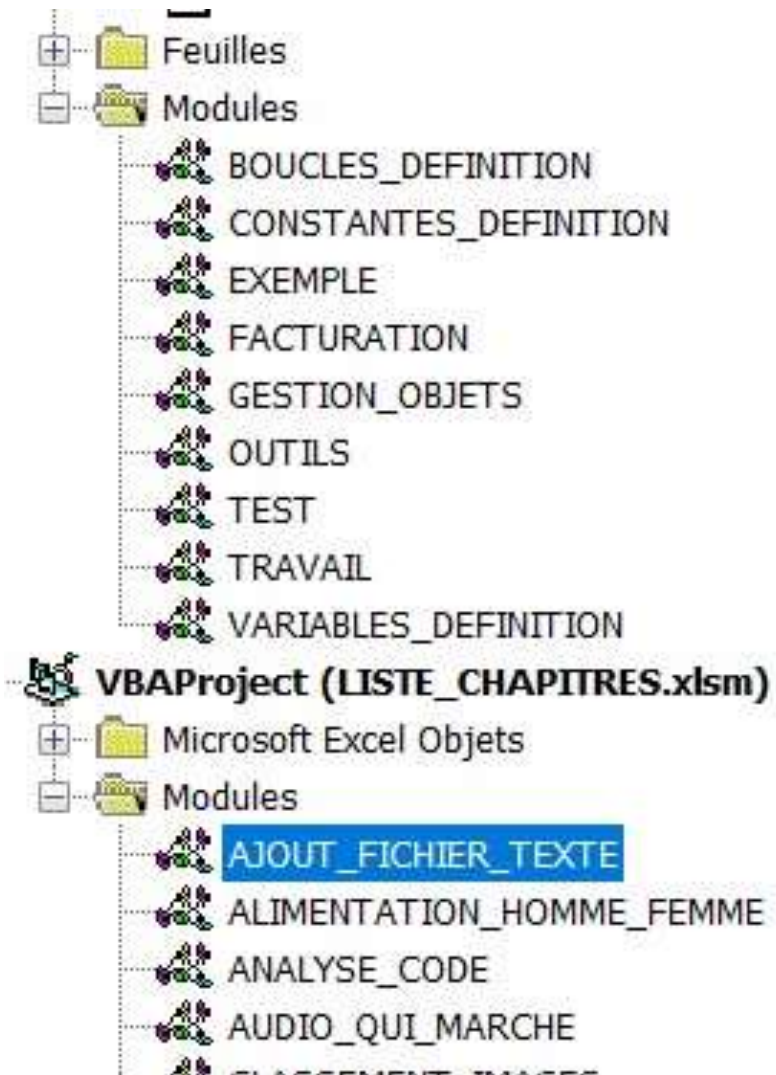
```
    Call ENREGISTREMENT
```

```
    Call VIDER_ZONE_SAISIE
```

```
End Sub
```


Organisation d'un module VBA

Le besoin d'organiser son code



Organisation d'un module VBA

Realisation de sa boite a outils

CRÉER SON PROPRE TYPE DE VARIABLE

Nous n'allons pas nous attarder sur ce point, voici juste un exemple :

```
'Création d'un type de variable
```

```
Type Utilisateur
```

```
    Nom As String
```

```
    Prenom As String
```

```
End Type
```

Organisation d'un module VBA

Realisation de sa boite a outils

```
Sub exemple()  
  
    'Déclaration  
    Dim user1 As Utilisateur  
  
    'Attributions des valeurs à user1  
    user1.Nom = "Smith"  
    user1.Prenom = "John"  
  
    'Exemple d'utilisation  
    MsgBox user1.Nom & " " & user1.Prenom  
  
End Sub
```

Organisation d'un module VBA

Realiser une formule en vba

```
Function RECUP_CALCUL_TVA(v_MONTANT_HT As Double) As Double
    Dim v_TVA As Double
    v_TVA = v_MONTANT_HT * 20 / 100
    RECUP_CALCUL_TVA = v_TVA
End Function
```

Sécurisation du code VBA

Sécurisation du code VBA

General

VBAProject - Propriétés du projet

Général Protection

Verrouillage du projet

☐ Verrouiller le projet pour l'affichage

Mot de passe pour l'affichage des propriétés du projet

Mot de passe

Confirmer le mot de

OK Annuler Aide

End Function

La methode Select

SÉLECTION D'UNE PLAGE DE CELLULES

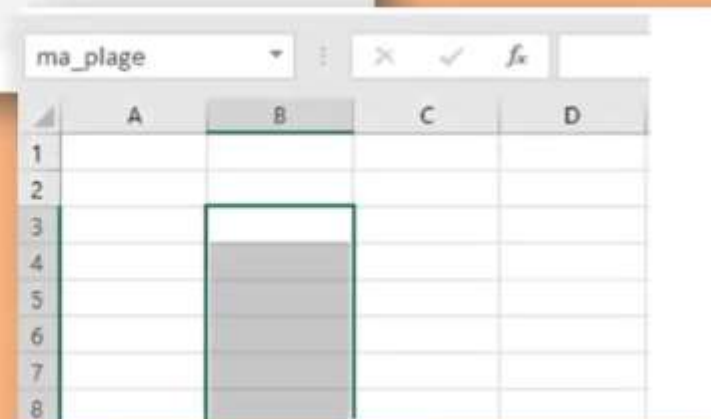
```
Sub exemple()  
  
    'Sélection des cellules A1 à A8  
    Range("A1:A8").Select  
  
End Sub
```


Utilisation avancée des collections, objets, méthodes et propriétés

La methode Select

SÉLECTION D'UNE PLAGE DE CELLULES NOMMÉE

```
Sub exemple()  
  
    'Sélection des cellules de la plage "ma_plage"  
    Range("ma_plage").Select  
  
End Sub
```



The screenshot shows an Excel spreadsheet with a named range 'ma_plage' selected. The range is located in column B, rows 3 to 7. The cells are highlighted in grey. The spreadsheet has columns A, B, C, and D, and rows 1 through 8. The formula bar at the top shows 'ma_plage'.

	A	B	C	D
1				
2				
3				
4				
5				
6				
7				
8				

Utilisation avancée des collections, objets, méthodes et propriétés

La methode Select

SÉLECTION D'UNE CELLULE EN FONCTION D'UN NUMÉRO DE LIGNE ET DE COLONNE

```
Sub exemple()  
  
    'Sélection de la cellule de la ligne 8 et de la colonne 1  
    Cells(8, 1).Select  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

La methode Select

```
Sub exemple()  
  
    'Sélection des lignes 2 à 6  
    Rows("2:6").Select  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

La methode Select

```
Sub exemple()  
  
    'Sélection des colonnes B à G  
    Columns("B:G").Select  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

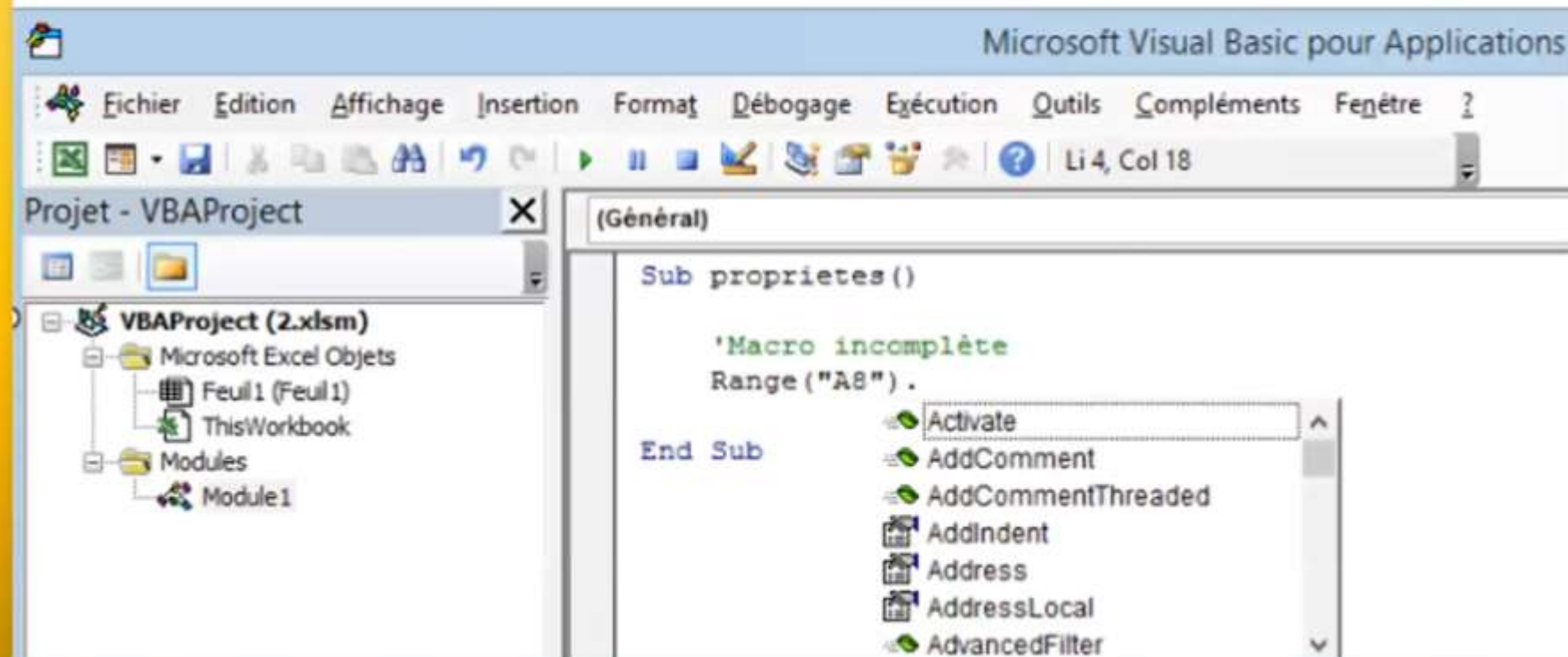
Les propriétés d'un classeur

```
Dim v_Classeur As Workbook  
Set v_FeuilleBase = ThisWorkbook.Worksheets("BASE_FONCTIONNALITES")
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés d'une cellule

Pour afficher la liste des possibilités que l'on peut associer à l'objet Range, ajoutez un `.` après `Range("A8")` :



Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés d'une cellule

Ce code permet de définir différentes propriétés à la cellule A8 de la feuille 2 :

```
Sub proprietes()  
  
    Sheets("Feuil2").Range("A8").Borders.Weight = 3  
    Sheets("Feuil2").Range("A8").Font.Bold = True  
    Sheets("Feuil2").Range("A8").Font.Size = 18  
    Sheets("Feuil2").Range("A8").Font.Italic = True  
    Sheets("Feuil2").Range("A8").Font.Name = "Arial"  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés d'une feuille

MODIFIER LES PROPRIÉTÉS D'UNE FEUILLE

```
Sub proprietes()  
  
    'Masquer une feuille  
    Sheets("Feuil3").Visible = 2  
  
    'Visible = -1 : afficher la feuille  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME DU TEXTE

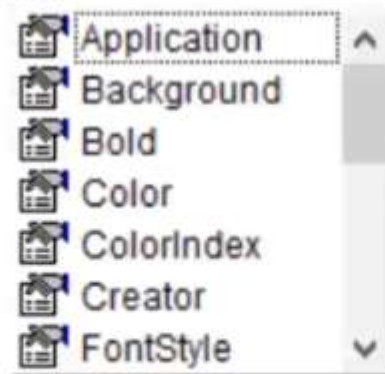
Après avoir sélectionné la propriété `Font` et ajouté un `.`, la liste des propriétés que l'on peut attribuer à la mise en forme du texte apparaît :

```
Sub proprietes()
```

```
    'Macro incomplète
```

```
    Range("A8").Font.
```

```
End Sub
```



Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME : TAILLE DU TEXTE

```
Sub proprietes()  
  
    'Modifier la taille du texte des cellules A1 à A8  
    Range("A1:A8").Font.Size = 18  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME : TEXTE EN GRAS

```
Sub proprietes()
```

```
    'Mettre en gras les cellules A1 à A8
```

```
    Range("A1:A8").Font.Bold = True
```

```
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME : TEXTE EN ITALIQUE

```
Sub proprietes()
```

```
    'Mettre en italique les cellules A1 à A8
```

```
    Range("A1:A8").Font.Italic = True
```

```
End Sub
```


Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME : TEXTE SOULIGNÉ

```
Sub proprietes()
```

```
    'Souligner les cellules A1 à A8
```

```
    Range("A1:A8").Font.Underline = True
```

```
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MISE EN FORME : POLICE

```
Sub proprietes()  
  
    'Modifier la police de caractères des cellules A1 à A8  
    Range("A1:A8").Font.Name = "Arial"  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

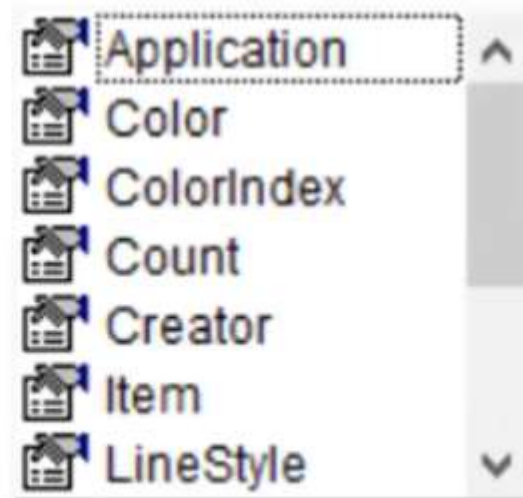
Les propriétés et méthodes d'un objet

AJOUTER DES BORDURES

```
Sub proprietes()
```

```
    'Ajouter une bordure aux cellules A1 à A8  
    Range("A1:A8").Borders.
```

```
End Sub
```



Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

```
Sub proprietes()  
  
    'Ajouter une bordure aux cellules A1 à A8  
    Range("A1:A8").Borders.Value = 1  
  
    'Value = 0 : pas de bordure  
  
End Sub
```

Utilisation avancée des collections, objets, méthodes et propriétés

Les propriétés et méthodes d'un objet

MODIFIER LA MISE EN FORME DE LA SÉLECTION ACTUELLE

```
Sub proprietes()  
  
    'Ajouter une bordure aux cellules sélectionnées  
    Selection.Borders.Value = 1  
  
End Sub
```

Manipulation des objets (classeur, feuille, plage)

les classes

```
Dim v_FeuilleFormulaire As Worksheet  
Set v_FeuilleFormulaire = ThisWorkbook.Worksheets("FORM LISTE FONCTIONNALITES ")  
Dim v_FeuilleBase As Worksheet  
Set v_FeuilleBase = ThisWorkbook.Worksheets("BASE_FONCTIONNALITES")  
  
v_FeuilleFormulaire.Select
```

```
v_Domaine = v_FeuilleFormulaire.Cells(1, 12).Value
```


Manipulation des objets (classeur, feuille, plage)

Les couleurs

```
Sub couleurs()
```

```
    Range("A1").Font.
```

```
End Sub
```



Manipulation des objets (classeur, feuille, plage)

Les couleurs

COLORINDEX

Voici les 56 couleurs disponibles avec `ColorIndex` :

	A	B	C	D	E	F	G	H	I
1	1	2	3	4	5	6	7	8	
2	9	10	11	12	13	14	15	16	
3	17	18	19	20	21	22	23	24	
4	25	26	27	28	29	30	31	32	
5	33	34	35	36	37	38	39	40	
6	41	42	43	44	45	46	47	48	
7	49	50	51	52	53	54	55	56	

Manipulation des objets (classeur, feuille, plage)

Les couleurs

Pour appliquer à notre texte l'une de ces 56 couleurs, nous écrirons :

```
Sub couleurs()  
  
    'Couleur du texte en A1 : vert (couleur 10)  
    Range("A1").Font.ColorIndex = 10  
  
End Sub
```

Manipulation des objets (classeur, feuille, plage)

Les couleurs

COLOR

Voici un exemple similaire avec `Color` :

```
Sub couleurs()  
  
    'Couleur du texte en A1 : RGB(50, 200, 100)  
    Range("A1").Font.Color = RGB(50, 200, 100)  
  
End Sub
```

Manipulation des objets (classeur, feuille, plage)

Les couleurs

RGB en français signifie **RVB** (Rouge Vert Bleu), les valeurs vont de 0 à 255 pour chaque couleur.

Quelques exemples de couleurs pour mieux comprendre :

- RGB(0, 0, 0) : noir
- RGB(255, 255, 255) : blanc
- RGB(255, 0, 0) : rouge
- RGB(0, 255, 0) : vert
- RGB(0, 0, 255) : bleu

Manipulation des objets (classeur, feuille, plage)

Les couleurs

```
Sub couleurs()  
  
    'Couleur du texte en A1 : RGB(192, 32, 255)  
    Range("A1").Font.Color = RGB(192, 32, 255)  
  
End Sub
```

Ce qui nous donne :

	A	B	C	D
1	Exemple			

Manipulation des objets (classeur, feuille, plage)

Les couleurs

CRÉER UNE BORDURE COLORÉE

Nous allons créer une macro qui va ajouter une bordure à la cellule active avec `ActiveCell`.

La bordure sera rouge et épaisse :

```
Sub couleurs()  
  
    'Epaisseur de la bordure  
    ActiveCell.Borders.Weight = 4  
  
    'Couleur de la bordure : rouge  
    ActiveCell.Borders.Color = RGB(255, 0, 0)  
  
End Sub
```

Aperçu :

	A	B	C
1			
2			
3			

Manipulation des objets (classeur, feuille, plage)

Les couleurs

COLORER LE FOND DES CELLULES SÉLECTIONNÉES

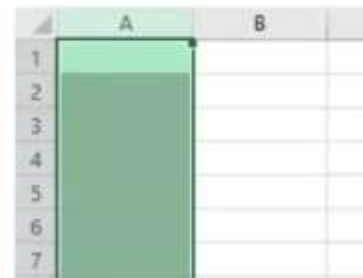
```
Sub couleurs()
```

```
'Colorer le fond des cellules sélectionnées
```

```
Selection.Interior.Color = RGB(174, 240, 194)
```

```
End Sub
```

Aperçu :



	A	B
1		
2		
3		
4		
5		
6		
7		

Manipulation des objets (classeur, feuille, plage)

Les couleurs

COLORER L'ONGLET D'UNE FEUILLE

```
Sub couleurs()  
  
    'Colorer l'onglet de la feuille "Feuil1"  
    Sheets("Feuil1").Tab.Color = RGB(255, 0, 0)  
  
End Sub
```



Manipulation des objets (classeur, feuille, plage)

Manipulation des objets (classeur, feuille, plage)

```
For v_NumLigne = 3 To v_derniereLigne
```

```
    Call INSERTION_LIGNE("BASE_FONCTIONNALITES", 3)
```

```
    v_FONCTIONNALITE = v_FeuilleFormulaire.Cells(v_NumLigne, 12).Value
```

```
    v_FeuilleBase.Cells(3, 1).Value = v_Domaine
```

```
    v_FeuilleBase.Cells(3, 2).Value = v_FONCTIONNALITE
```

```
    v_FeuilleBase.Cells(3, 5).Value = "1 ==> A FAIRE"
```

```
Next v_NumLigne
```

```
v_FeuilleFormulaire.Select
```

```
v_FeuilleFormulaire.Cells(1, 12).Value = ""
```

Manipulation des objets (classeur, feuille, plage)

Selectionner une cellule

`v_FeuilleFormulaire.Select`

`v_FeuilleFormulaire.Cells(7, 7).Select`

Affectation d'objets et de variables d'objets

Affectation de valeur a une cellule

Nous voulons maintenant donner la valeur 48 à la cellule A8 :

```
Sub proprietes()
```

```
    'Cellule A8 = 48
```

```
    Range("A8").Value = 48
```

```
    'Traduction :
```

```
    'La valeur de la cellule A8 est désormais : 48
```

```
End Sub
```

Affectation d'objets et de variables d'objets

Affectation de valeur a une cellule

Puis, la valeur `Exemple de texte` à A8 (le texte doit être mis entre `" "`)

```
Sub proprietes()
```

```
    'Cellule A8 = Exemple de texte
```

```
    Range("A8").Value = "Exemple de texte"
```

```
End Sub
```


Affectation d'objets et de variables d'objets

Affectation de valeur a une variable

Attribuons ensuite les valeurs des cellules aux variables :

```
Sub variables()  
  
    'Déclaration des variables  
    Dim nom As String, prenom As String, age As Integer  
  
    'Valeurs des variables  
    nom = Cells(2, 1)  
    prenom = Cells(2, 2)  
    age = Cells(2, 3)  
  
End Sub
```

Affectation d'objets et de variables d'objets

Affectation de valeur a une variable

```
Sub variables()  
  
    'Déclaration des variables  
    Dim nom As String, prenom As String, age As Integer  
  
    'Valeurs des variables  
    nom = Cells(2, 1)  
    prenom = Cells(2, 2)  
    age = Cells(2, 3)  
  
    'Boîte de dialogue  
    MsgBox nom & " " & prenom & ", " & age & " ans"  
  
End Sub
```

Affectation d'objets et de variables d'objets

Affectation de valeur a une variable

Ce qui nous donne :

	A	B	C	D	E	F
1	Nom	Prénom	Age			
2	Smith	John	40			
3	Smith	John	41			
4	Smith	John	42			
5	Smith	John	43			
6	Smith	John	44			
7	Smith	John	45			
8	Smith	John	46			
9	Smith	John	47			
10	Smith	John	48			
11	Smith	John	49			
12	Smith	John	50			

Bouton

Microsoft Excel

Smith John, 40 ans

OK

Affectation d'objets et de variables d'objets

Recuperation du contenu d'une cellule

```
v_Domaine = v_FeuilleFormulaire.Cells(7, 7).Value
```

```
v_FONCTIONNALITE = v_FeuilleFormulaire.Cells(11, 7).Value
```

Introduction au code Visual Basic

Difference entre une variable et un parametre

LES ARGUMENTS

Les arguments permettent de transmettre des valeurs d'une procédure à une autre (car rappelez-vous que par défaut les variables ne sont pas accessibles depuis les autres procédures).

Introduction au code Visual Basic

Difference entre une variable et un parametre

Ajout d'un argument `texte` à la procédure `avertissement` :

```
Private Sub avertissement(texte As String)
```

```
    MsgBox "Attention : " & texte & " !"
```

```
End Sub
```

```
Sub exemple()
```

```
    If Range("A1") = "" Then 'Si A1 est vide  
        avertissement "cellule vide"
```

```
    ElseIf Not IsNumeric(Range("A1")) Then 'Si A1 est non numérique  
        avertissement "valeur non numérique"
```

```
    End If
```

```
End Sub
```

```
avertissement "valeur non numérique"
```

Introduction au code Visual Basic

Difference entre une variable et un parametre

L'argument ajouté à la procédure `avertissement` est de type `String` :

```
Private Sub avertissement(texte As String)
```

Pour exécuter la procédure `avertissement`, il faudra donc entrer en argument une valeur de type `String` :

```
avertissement "cellule vide"
```

En cas d'arguments multiples, ceux-ci doivent être séparés par des virgules.

Introduction au code Visual Basic

Difference entre une variable et un parametre

LES ARGUMENTS OPTIONNELS

Par défaut, si une procédure requiert des arguments, ceux-ci sont obligatoires pour exécuter la procédure.

```
Private Sub boiteDialogue(nom As String, Optional prenom, Optional age)
```

'Exemple 1 : on affiche le nom

```
boiteDialogue nom
```

'Exemple 2 : on affiche le nom et le prénom

```
boiteDialogue nom, prenom
```

'Exemple 3 : on affiche le nom et l'âge

```
boiteDialogue nom, , age
```

'Exemple 4 : on affiche le nom, le prénom et l'âge

```
boiteDialogue nom, prenom, age
```

Optional prenom

Introduction au code Visual Basic

Difference entre une variable et un parametre

Les arguments doivent être indiqués dans l'ordre.

Introduction au code Visual Basic

Difference entre une variable et un parametre

```
Private Sub boiteDialogue(nom As String, Optional prenom, Optional age)

    'Si l'âge est manquant
    If IsMissing(age) Then

        If IsMissing(prenom) Then 'Si le prénom est manquant, on n'affiche que le nom
            MsgBox nom
        Else 'Sinon, on affiche le nom et le prénom
            MsgBox nom & " " & prenom
        End If

        'Si l'âge a été renseigné
    Else

        If IsMissing(prenom) Then 'Si le prénom est manquant, on affiche le nom et l'âge
            MsgBox nom & ", " & age & " ans"
        Else 'Sinon on affiche le nom, le prénom et l'âge
            MsgBox nom & " " & prenom & ", " & age & " ans"
        End If

    End If

End Sub
```

Introduction au code Visual Basic

Difference entre une variable et un parametre

Il est également possible de renseigner des valeurs par défaut aux arguments optionnels et de tester ensuite ces valeurs (au lieu d'utiliser la fonction IsMissing) :

Introduction au code Visual Basic

Difference entre une variable et un parametre

```
Private Sub boiteDialogue(nom As String, Optional prenom As String = "", Optional age As Integer = 0)
```

Introduction au code Visual Basic

Difference entre une variable et un parametre

```
'Si l'âge est manquant
If age = 0 Then

    If prenom = "" Then 'Si le prénom est manquant, on n'affiche que le nom
        MsgBox nom
    Else 'Sinon, on affiche le nom et le prénom
        MsgBox nom & " " & prenom
    End If

'Si l'âge a été renseigné
```

Introduction au code Visual Basic

Difference entre une variable et un parametre

```
'Si l'âge a été renseigné
Else

    If prenom = "" Then 'Si le prénom est manquant, on n'affiche que le nom
        MsgBox nom & ", " & age & " ans"
    Else 'Sinon on affiche le nom, le prénom et l'âge
        MsgBox nom & " " & prenom & ", " & age & " ans"
    End If

End If

End Sub
```


Introduction au code Visual Basic

Difference entre une variable et un parametre

ByRef

BYREF - BYVAL

Par défaut, les arguments sont de type `ByRef` ce qui signifie que, si une variable est passée en argument, c'est sa référence qui est transmise. Autrement dit, si la variable est modifiée dans la sous-procédure, elle le sera également dans la procédure d'appel.

Introduction au code Visual Basic

Difference entre une variable et un parametre

ByRef

```
Sub exemple()  
  
    Dim nombre As Integer  
    nombre = 30  
  
    carre nombre  
  
    MsgBox nombre  
  
End Sub
```

```
Private Sub carre(ByRef valeur As Integer)  
    'par défaut)  
  
    valeur = valeur ^ 2  
  
End Sub
```

'Il n'est pas nécessaire de préciser ByRef

par défaut)

Introduction au code Visual Basic

Difference entre une variable et un parametre

ByVal

La seconde possibilité consiste à utiliser **ByVal** .

Contrairement à **ByRef** qui fait directement référence à la variable, **ByVal** transmet uniquement sa valeur, ce qui signifie que la variable passée en argument ne subit aucune modification.

Voici ce qui se passe avec le code précédent et **ByVal** :

Introduction au code Visual Basic

Difference entre une variable et un parametre

ByVal

'La valeur initiale de la variable "nombre" est 30

nombre = 30

'La sous-procédure est lancée avec la variable "nombre" en argument

carre nombre

Introduction au code Visual Basic

Difference entre une variable et un parametre

ByVal

'La variable "valeur" copie la valeur de la variable "nombre" (les 2 variables ne sont pas liées)

```
Private Sub carre(ByVal valeur As Integer)  
,
```

'La valeur de la variable "valeur" est modifiée

```
valeur = valeur ^ 2
```

'Fin de la sous-procédure (dans cet exemple, la sous-procédure n'aura servi à rien)

```
End Sub
```

'La variable "nombre" n'a pas été modifiée, 30 est donc affiché dans la boîte de dialogue

```
MsgBox nombre
```

Introduction au code Visual Basic

Le mot clef FUNCTION

FONCTION ISNUMERIC

```
If IsNumeric(Range("A1")) = False Then
```

Dans le cas où nous voulons vérifier si la valeur n'est pas numérique, nous avons également deux possibilités :

```
If IsNumeric(Range("A1")) = False Then 'Si la valeur n'est pas numérique
```

```
If Not IsNumeric(Range("A1")) Then 'Si la valeur n'est pas numérique
```

Introduction au code Visual Basic

Le mot clef FUNCTION

FONCTIONS DE DATES

La fonction **IsDate** renvoie True si la valeur est une date ou False si ce n'est pas le cas :

```
If IsDate(Range("A1")) Then 'Si la valeur est une date
```

La fonction **Day** permet d'extraire le jour d'une date :

```
If Day(Range("A1")) = 1 Then 'Si c'est le premier jour du mois
```


Introduction au code Visual Basic

Le mot clef FUNCTION

La fonction **Year** permet d'extraire l'année d'une date :

```
If Year(Range("A1")) = 2024 Then 'Si c'est une date de l'année 2024
```

Introduction au code Visual Basic

Le mot clef FUNCTION

La fonction **Weekday** renvoie le numéro du jour de la semaine :

```
If Weekday(Range("A1"), 2) >= 6 Then 'Si c'est un samedi ou un dimanche
```

Introduction au code Visual Basic

Le mot clef FUNCTION

La fonction **Date** renvoie la date actuelle :

```
If Range("A1") < Date Then 'Si la date est passée
```

Introduction au code Visual Basic

Le mot clef FUNCTION

Dans cet exemple, la condition est vraie car aucun type ni valeur n'ont été attribués à `maVariable` :

```
Sub exemple()  
  
    Dim maVariable  
  
    If IsEmpty(maVariable) Then  
        MsgBox "Ma variable n'a pas été initialisée !"  
    Else  
        MsgBox "Ma variable contient : " & maVariable  
    End If  
  
End Sub
```

Introduction au code Visual Basic

Le mot clef SUB

```
Sub COPIE()  
    Selection.Copy  
End Sub
```

```
Sub COLLAGE()  
    ActiveSheet.Paste  
End Sub
```

Introduction au code Visual Basic

Le mot clef WITH

Nous pouvons utiliser `With` pour éviter les répétitions de `Sheets("Feuil2").Range("A8")` :

```
Sub proprietes()  
  
    'Début de l'instruction avec : With  
    With Sheets("Feuil2").Range("A8")  
        .Borders.Weight = 3  
        .Font.Bold = True  
        .Font.Size = 18  
        .Font.Italic = True  
        .Font.Name = "Arial"  
    'Fin de l'instruction avec : End With  
    End With  
  
End Sub
```

Introduction au code Visual Basic

Le mot clef WITH

```
Sub proprietes()  
  
    With Sheets("Feuil2").Range("A8")  
        .Borders.Weight = 3  
        With .Font  
            .Bold = True  
            .Size = 18  
            .Italic = True  
            .Name = "Arial"  
        End With  
    End With  
  
End Sub
```


Introduction au code Visual Basic

Les mots clefs d'une macro

```
Sub Macro1()
```

```
End Sub
```

Introduction au code Visual Basic

Les obligations relatives aux noms utilisés

```
Sub VALIDATION()
```

```
    Call INSERTION_LIGNE_11
```

```
    Call ENREGISTREMENT
```

```
    Call VIDER_ZONE_SAISIE
```

```
End Sub
```

Types et portées des variables

les parametres optionnels

```
Sub DireBonjour(Optional Nom As Variant)
    If IsMissing(Nom) Then
        MsgBox "Bonjour, Inconnu!"
    Else
        MsgBox "Bonjour, " & Nom & "!"
    End If
End Sub
```

Types et portées des variables

Les tableaux de variables

LE TABLEAU À 1 DIMENSION

'Exemple de déclaration de tableau à 1 dimension

```
Dim tab1(4) As String
```

0

1

2

3

4

Types et portées des variables

Les tableaux de variables

La première case d'un tableau est le 0.

```
tab1(0) = "Valeur de la case 0"  
tab1(1) = "Valeur de la case 1"  
tab1(2) = "Valeur de la case 2"  
tab1(3) = "Valeur de la case 3"  
tab1(4) = "Valeur de la case 4"
```

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

Les tableaux permettent de stocker un grand nombre de valeurs contrairement aux variables qui ne peuvent stocker qu'une seule valeur à la fois.

Nous avons effleuré le sujet à la leçon sur les **variables**, nous allons maintenant l'approfondir.

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

DÉCLARATION D'UN TABLEAU

Voici quelques exemples de déclarations (si les 2 premières déclarations ne sont pas claires pour vous, relisez ceci) :

```
'Exemple de déclaration d'un tableau à 1 dimension
```

```
Dim tab1(4)
```

```
'Exemple de déclaration d'un tableau à 2 dimensions
```

```
Dim tab2(6, 1)
```

```
'Exemple de déclaration d'un tableau dynamique
```

```
Dim tab3()
```


Types et portées des variables

Les tableaux de variables

Si vous ne pouvez pas entrer de valeurs fixes (parce que cela dépend de la taille de la base de données par exemple), laissez les parenthèses vides.

Vous n'avez pas besoin de déclarer un type (String, Integer, etc.) car dans bien des cas cela ralentirait votre procédure.

ENREGISTRER DES DONNÉES DANS UN TABLEAU

Commençons par enregistrer ces quelques données dans un tableau :

	A	B	C
1	Date		
2	11.03.2026		
3	23.10.2018		
4	21.03.2021		
5	08.03.2023		
6	05.04.2016		
7	22.03.2018		
8	27.07.2021		
9	25.09.2025		
10	04.02.2016		
11	11.09.2019		
12	25.01.2020		
13			
14			

LES TABLEAUX

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

```
Dim tableau(10)
```

Nous voulons enregistrer ici 11 x 1 valeurs, il faudra donc déclarer un tableau à une dimension :

```
Dim tableau(10)
```

Le tableau `tableau(10)` peut contenir 11 valeurs, car rappelez-vous que la numérotation d'un tableau commence à 0.

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

Chaque élément du tableau reçoit ensuite sa valeur :

```
Sub exemple()  
  
    Dim tableau(10)  
  
    'Enregistrement des valeurs dans le tableau  
    tableau(0) = Range("A2")  
    tableau(1) = Range("A3")  
    tableau(2) = Range("A4")  
    tableau(3) = Range("A5")  
    tableau(4) = Range("A6")  
    tableau(5) = Range("A7")  
    tableau(6) = Range("A8")  
    tableau(7) = Range("A9")  
    tableau(8) = Range("A10")  
    tableau(9) = Range("A11")  
    tableau(10) = Range("A12")  
  
End Sub
```

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

Mais pour éviter toutes ces répétitions, l'utilisation d'une boucle **For** est plus que recommandée

```
Sub exemple()  
  
    Dim tableau(10), i As Integer  
  
    'Enregistrement des valeurs dans le tableau  
    For i = 0 To 10  
        tableau(i) = Range("A" & i + 2)  
    Next  
  
End Sub
```

Types et portées des variables

Les tableaux de variables

LES TABLEAUX

```
Sub exemple()  
  
    Dim tableau(10), i As Integer  
  
    'Enregistrement des valeurs dans le tableau  
    For i = 0 To 10  
        tableau(i) = Range("A" & i + 2)  
    Next  
  
    'Affichage 1  
    MsgBox tableau(3) 'Renvoie : 08.03.2023  
  
    'Modification de l'une des valeurs  
    tableau(3) = Year(tableau(3))  
  
    'Affichage 2  
    MsgBox tableau(3) 'Renvoie : 2023  
  
End Sub
```

Notez que vous pouvez utiliser, modifier chaque élément du tableau individuellement comme une variable.

En voici un exemple avec `tableau(3)`, la 4^e valeur du tableau :

Dans cet exemple, la fonction Year (qui renvoie l'année d'une date) a été utilisée pour modifier `tableau(3)`.

Types et portées des variables

Les variables complexes et le mot clef Set

'Exemple : objet (objet Worksheet pour cet exemple)

```
Dim varFeuille As Worksheet
```

```
Set varFeuille = Sheets("Feuil2") 'Set => attribution d'une valeur à une variable objet
```

'Exemple d'utilisation de la variable objet : activation de la feuille

```
varFeuille.Activate
```

Types et portées des variables

Les variables et constantes et le mot clef Dim

```
Dim maVariable As Integer
```

- **Dim** : déclaration de la variable
- **maVariable** : nom choisi pour cette variable (sans espaces)
- **As** : déclaration du type de la variable
- **Integer** : type de la variable

Types et portées des variables

Les variables et constantes et le mot clef Dim

Une valeur est ensuite donnée à cette variable :

```
maVariable = 12
```

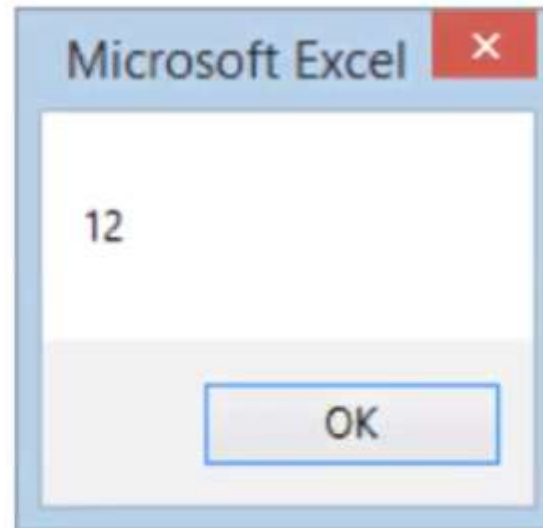
Types et portées des variables

Les variables et constantes et le mot clef Dim

Et enfin, la valeur de la variable est affichée dans une boîte de dialogue :

```
MsgBox maVariable
```

Le résultat de ce code :



Types et portées des variables

Les variables et constantes et le mot clef Dim

Nom	Type	Détails
Byte	Numérique	Nombre entier de 0 à 255.
Integer	Numérique	Nombre entier de -32'768 à 32'767.
Long	Numérique	Nombre entier de - 2'147'483'648 à 2'147'483'647.
Currency	Numérique	Nombre à décimale fixe de -922'337'203'685'477.5808 à 922'337'203'685'477.5807.
Single	Numérique	Nombre à virgule flottante de -3.402823E38 à 3.402823E38.
Double	Numérique	Nombre à virgule flottante de -1.79769313486232E308 à 1.79769313486232E308.

Types et portées des variables

Les variables et constantes et le mot clef Dim

String	Texte	Texte.
Date	Date	Date et heure.
Boolean	Boolean	True (vrai) ou False (faux).
Object	Objet	Objet.
Variant	Tous	Tout type de données (type par défaut si la variable n'est pas déclarée).

Types et portées des variables

Les variables et constantes et le mot clef Dim

Quelques exemples avec différents types :

'Exemple : nombre entier

```
Dim nbEntier As Integer
```

```
nbEntier = 12345
```

'Exemple : nombre à virgule

```
Dim nbVirgule As Single
```

```
nbVirgule = 123.45
```

Types et portées des variables

Les variables et constantes et le mot clef Dim

```
'Exemple : date  
Dim varDate As Date  
varDate = "15/07/2024"
```

```
'Exemple : vrai/faux  
Dim varBoolean As Boolean  
varBoolean = True
```

```
'Exemple : objet (objet Worksheet pour cet exemple)  
Dim varFeuille As Worksheet  
Set varFeuille = Sheets("Feuil2") 'Set => attribution d'une valeur à une variable objet
```

Types et portées des variables

Les variables et constantes et le mot clef Dim

```
Sub variables()  
  
    'Déclaration des variables  
    Dim nom As String, prenom As String, age As Integer  
  
End Sub
```


Types et portées des variables

Les variables et constantes et le mot clef Dim

LES CONSTANTES

```
Sub exemple()
```

```
Cells(1, 1) = Cells(1, 2) * 0.1234
```

```
Cells(2, 1) = Cells(2, 2) * 0.1234
```

```
Cells(3, 1) = Cells(3, 2) * 0.1234
```

```
Cells(4, 1) = Cells(4, 2) * 0.1234
```

```
Cells(5, 1) = Cells(5, 2) * 0.1234
```

```
End Sub
```

Types et portées des variables

Portee des variables : variable privée

LA PORTÉE DES VARIABLES

Si la variable est déclarée au début d'une procédure (Sub), elle ne peut être utilisée que dans cette même procédure. La valeur de la variable n'est pas conservée après l'exécution de la procédure.

Types et portées des variables

Portee des variables : variable privée

```
Sub procedure1()
```

```
    Dim var1 As Integer
```

```
    '=> Utilisation de la variable dans la procédure uniquement
```

```
End Sub
```

```
Sub procedure2()
```

```
    '=> Impossible d'utiliser var1 ici
```

```
End Sub
```

Types et portées des variables

Portee des variables : variable privée

```
Dim var1 As Integer

Sub procedure1()

    '=> Utilisation de var1 possible

End Sub

Sub procedure2()

    '=> Utilisation de var1 possible

End Sub
```

Types et portées des variables

Portee des variables : variable public

```
Public var1 As Integer
```

Déclaration des constantes

Les variables et constantes et le mot clef Dim

```
Sub exemple()
```

```
'Déclaration de la constante + attribution de sa valeur
```

```
Const TAUX_TVA As Double = 0.1234
```

```
Cells(1, 1) = Cells(1, 2) * TAUX_TVA
```

```
Cells(2, 1) = Cells(2, 2) * TAUX_TVA
```

```
Cells(3, 1) = Cells(3, 2) * TAUX_TVA
```

```
Cells(4, 1) = Cells(4, 2) * TAUX_TVA
```

```
Cells(5, 1) = Cells(5, 2) * TAUX_TVA
```

```
End Sub
```

Utilisation des instructions de boucles

Imbrication des boucles for next

```
Sub BouclesImbriquees()  
    Dim i As Integer  
    Dim j As Integer  
  
    For i = 1 To 3  
        For j = 1 To 2  
            MsgBox "i = " & i & ", j = " & j  
        Next j  
    Next i  
End Sub
```


Utilisation des instructions de boucles

la boucle DO WHILE LOOP

```
Sub exemple()
```

```
Cells(1, 1) = 1  
Cells(2, 1) = 2  
Cells(3, 1) = 3  
Cells(4, 1) = 4  
Cells(5, 1) = 5  
Cells(6, 1) = 6  
Cells(7, 1) = 7  
Cells(8, 1) = 8  
Cells(9, 1) = 9  
Cells(10, 1) = 10  
Cells(11, 1) = 11  
Cells(12, 1) = 12
```

```
End Sub
```

Ce code est très répétitif ...

```
Do While numero <= 12 'Tant que  
    Cells(numero, 1) = numero 'M  
    numero = numero + 1 'Le nume  
Loop
```

Voici la macro répétitive ci-dessus avec la boucle Do :

```
Sub exemple()
```

```
    Dim numero As Integer
```

```
    numero = 1 'Numéro de départ
```

```
    Do While numero <= 12 'Tant que la variable numero est <= 12, la boucle est répétée
```

```
        Cells(numero, 1) = numero 'Numérotation
```

```
        numero = numero + 1 'Le numéro est augmenté de 1 à chaque boucle
```

```
    Loop
```

```
End Sub
```

Utilisation des instructions de boucles

la boucle DO WHILE LOOP

Voici la boucle **Do** :

```
Sub exemple()  
  
    Do While [CONDITION]  
        'Instructions  
    Loop  
  
End Sub
```

Utilisation des instructions de boucles

la boucle DO WHILE LOOP

Avec `Do`, la condition peut également être placée en fin de boucle, ce qui implique que les instructions seront dans tous les cas exécutées au moins une fois :

```
Sub exemple()  
  
    Do  
        'Instructions  
    Loop While [CONDITION]  
  
End Sub
```

Utilisation des instructions de boucles

la boucle DO WHILE LOOP

Plutôt que de répéter la boucle tant que la condition est vraie, il est possible de quitter la boucle lorsque la condition est vraie en remplaçant `While` par `Until` :

```
Sub exemple()  
  
    Do Until [CONDITION]  
        'Instructions  
    Loop  
  
End Sub
```

Utilisation des instructions de boucles

La boucle FOR NEXT

FOR NEXT

La boucle **For** est répétée ici 5 fois.

```
Sub exemple()  
  
    Dim i As Integer  
  
    For i = 1 To 5  
        'Instructions  
    Next  
  
End Sub
```

Utilisation des instructions de boucles

La boucle FOR NEXT

A chaque répétition de la boucle, la variable `i` est automatiquement incrémentée de 1 :

```
Sub exemple()  
  
    Dim i As Integer  
  
    For i = 1 To 5  
        MsgBox i 'Renvoie les valeurs : 1 / 2 / 3 / 4 / 5  
    Next  
  
End Sub
```

Utilisation des instructions de boucles

La boucle FOR NEXT

Si nécessaire, vous pouvez modifier l'incrément (par défaut à 1) de la boucle en ajoutant **Step**

```
Sub exemple()  
  
    Dim i As Integer  
  
    For i = 10 To 0 Step -2  
        MsgBox i 'Renvoie les valeurs : 10 / 8 / 6 / 4 / 2 / 0  
    Next  
  
End Sub
```


Utilisation des instructions de boucles

la boucle FOREACH

FOR EACH NEXT

La boucle **For Each** permet de parcourir chaque élément d'un ensemble d'éléments, par exemple parcourir chaque cellule d'une plage de cellules :

```
Sub exemple()  
  
    Dim cellule As Range  
  
    For Each cellule In Range("A1:B3")  
        cellule = cellule.Address  
    Next  
  
End Sub
```

Utilisation des instructions de boucles

la boucle FOREACH

Parcourir chaque feuille du classeur :

```
Sub exemple()  
  
    Dim feuille As Worksheet  
  
    For Each feuille In Worksheets  
        MsgBox feuille.Name  
    Next  
  
End Sub
```

Utilisation des instructions de boucles

la boucle FOREACH

Parcourir chaque élément d'un tableau :

```
Sub exemple()  
  
    Dim tableau(2) As String  
  
    tableau(0) = "A"  
    tableau(1) = "B"  
    tableau(2) = "C"  
  
    For Each valeur In tableau  
        MsgBox valeur  
    Next  
  
End Sub
```

Utilisation des instructions de boucles

la boucle WHILE WEND

```
Sub ExempleWhileWend()  
    Dim i As Integer  
    i = 1  
  
    While i <= 5  
        MsgBox "La valeur de i est : " & i  
        i = i + 1  
    Wend  
End Sub
```

Utilisation des instructions de boucles

Sortie prematuree d'une boucle

QUITTER UNE BOUCLE PRÉMATURÉMENT

Il est possible de quitter une boucle `For` prématurément grâce à l'instruction suivante :

```
Exit For 'Quitter une boucle For
```

`Exit For`

Utilisation des instructions de boucles

Sortie prematuree d'une boucle

```
Sub exemple()  
  
    Dim i As Integer  
  
    'Boucle pour 100 lignes au maximum  
    For i = 1 To 100  
  
        'Si la cellule vaut 1  
        If Cells(i, 1) = 1 Then 'Si l'objectif est atteint  
            MsgBox "La cellule a été trouvée à la ligne " & i & " !"   
            Exit For 'On quitte la boucle For  
        End If  
  
    Next  
  
End Sub
```

Exit For

Utilisation des instructions de boucles

Sortie prematuree d'une boucle

Les autres instructions **Exit** :

```
Exit Do 'Quitter une boucle Do
```

```
Exit Sub 'Quitter une procédure
```

```
Exit Function 'Quitter une fonction
```


Utilisation des instructions de boucles

Traitement dans une boucle

les boucles

Les boucles permettent de répéter des instructions un certain nombre de fois pour vous éviter de devoir écrire des macros d'une longueur interminable et vous faire gagner un temps considérable.

Utilisation des instructions de boucles

Trouver la dernière colonne remplie d'une ligne

```
Function DerniereColonneRemplie(ligne As Long) As Long
    Dim derniereCol As Long

    ' On utilise la fonction Cells pour cibler la cellule (ligne, colonne)
    ' On utilise "End(xlToLeft)" pour trouver la dernière colonne non vide

    derniereCol = Cells(ligne, Columns.Count).End(xlToLeft).Column

    ' On retourne le numéro de la colonne
    DerniereColonneRemplie = derniereCol
End Function
```

Utilisation des instructions de boucles

Trouver la dernière ligne remplie d'une colonne

```
Sub TrouverDerniereLigne()  
    Dim DerniereLigne As Long  
    DerniereLigne = Cells(Rows.Count, 1).End(xlUp).Row  
    MsgBox "La dernière ligne remplie dans la colonne A est : " & DerniereLigne  
End Sub
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

La principale instruction est **If** , voici comment fonctionne :

```
If [CONDITION] Then '=> SI condition vraie  
ALORS  
    'Instructions si vrai  
Else '=> SINON (facultatif)  
    'Instructions si faux  
End If
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

les opérateurs de comparaison :

=	Est égal à
<>	Est différent de
<	Est plus petit que
<=	Est plus petit ou égal à
>	Est plus grand que
>=	Est plus grand ou égal à

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

And	Et	[CONDITION 1] And [CONDITION 2] Les 2 conditions doivent être vraies
Or	Ou	[CONDITION 1] Or [CONDITION 2] Au moins 1 des 2 conditions doit être vraie
Not	Faux	Not [CONDITION] La condition doit être fausse
Mod	Modulo	[NOMBRE] Mod [DIVISEUR] Cet opérateur retourne le reste d'une division

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

ELSEIF

`ElseIf` permet d'ajouter plusieurs conditions à la suite :

```
If [CONDITION 1] Then '=> SI la condition 1 est vraie ALORS
    'Instructions 1
ElseIf [CONDITION 2] Then '=> SINON, SI la condition 2 est vraie ALORS
    'Instructions 2
Else '=> SINON
    'Instructions 3
End If
```


Condition, itération, compteur, choix multiple

Les conditions avec if et end if

```
Sub commentaires()  
  
    'Variables  
    Dim note As Single, commentaire As String  
    note = Range("A1")  
  
    'Commentaire en fonction de la note  
    If note = 6 Then  
        commentaire = "Excellent résultat !"  
    ElseIf note >= 5 Then  
        commentaire = "Bon résultat"  
    ElseIf note >= 4 Then  
        commentaire = "Résultat satisfaisant"  
    ElseIf note >= 3 Then  
        commentaire = "Résultat insatisfaisant"  
    ElseIf note >= 2 Then  
        commentaire = "Mauvais résultat"  
    ElseIf note >= 1 Then  
        commentaire = "Résultat exécrable"  
    Else  
        commentaire = "Aucun résultat"  
    End If  
  
    'Commentaire en B1  
    Range("B1") = commentaire  
  
End Sub
```

```
Sub commentaires()  
  
    'Variables  
    Dim note As Single, commentaire As String  
    note = Range("A1")  
  
    'Commentaire en fonction de la note  
    If note = 6 Then  
        commentaire = "Excellent résultat !"  
    ElseIf note >= 5 Then  
        commentaire = "Bon résultat"  
  
    'Commentaire en B1  
    Range("B1") = commentaire
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

Jusque-là nous n'avons vu que cela :

```
maVariable = "Exemple 12345"
```

```
If maVariable = "Exemple 12345" Then '=> Vrai
```

Le caractère ***** peut remplacer : aucun, un ou plusieurs caractères :

```
maVariable = "Exemple 12345"
```

```
If maVariable Like "*12345*" Then '=> Vrai
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

Le caractère #

Le caractère # peut remplacer un caractère numérique de 0 à 9 :

```
maVariable = "Exemple 12345"
```

```
If maVariable Like "Exemple 12###" Then '=> Vrai
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

Le caractère ?

Le caractère ? peut remplacer un caractère quelconque :

```
maVariable = "Exemple 12345"
```

```
If maVariable Like "?xemple?1234?" Then '=> Vrai
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

- [abc] : remplace un des caractères suivants : a b c
- [a-g] : remplace un des caractères suivants : a b c d e f g
- [369] : remplace un des caractères suivants : 3 6 9
- [2-5] : remplace un des caractères suivants : 2 3 4 5
- [?*#] : remplace un des caractères suivants : ? * #

```
maVariable = "Exemple 12345"
```

```
If maVariable Like "[BIEN]xemple 1234[4-7]" Then '=> Vrai
```

Condition, itération, compteur, choix multiple

Les conditions avec if et end if

Pour remplacer un caractère non compris dans les valeurs entre crochets, un `!` doit être ajouté après `[`

, un `!` doit être ajouté après `[`

```
maVariable = "Exemple 12345"
```

```
If maVariable Like "[!FAUX]xemple 1234[!6-9]" Then '=> Vrai
```


Condition, itération, compteur, choix multiple

les conditions avec select case et end select

Voici la même macro avec **Select** :

```
Sub commentaires()  
  
    'Variables  
    Dim note As Single, commentaire As String  
    note = Range("A1")  
  
    'Commentaire en fonction de la note  
    Select Case note ' <= la valeur à tester (ici, la note)  
        Case Is = 6  
            commentaire = "Excellent résultat !"  
        Case Is >= 5  
            commentaire = "Bon résultat"  
        Case Is >= 4  
            commentaire = "Résultat satisfaisant"  
        Case Is >= 3  
            commentaire = "Résultat insatisfaisant"  
        Case Is >= 2  
            commentaire = "Mauvais résultat"  
        Case Is >= 1  
            commentaire = "Résultat exécration"  
        Case Else  
            commentaire = "Aucun résultat"  
    End Select
```

```
'Commentaire en fonction de la note  
Select Case note ' <= la valeur à tester (ici, la note)  
    Case Is = 6  
        commentaire = "Excellent résultat !"  
    Case Is >= 5  
        commentaire = "Bon résultat"
```

```
    Case Else  
        commentaire = "Aucun résultat"  
End Select
```


Condition, itération, compteur, choix multiple

les conditions avec select case et end select

Notez que nous pouvons également entrer plusieurs valeurs :

```
Case Is = 6, 7 'Si la valeur = 6 ou 7
```

```
Case Is <> 6, 7 'Si la valeur est différente de 6 ou 7
```

Ou une plage de valeurs :

```
Case 6 To 10 'Si la valeur = de 6 à 10
```

Utilisation des boîtes de dialogue prédéfinies

La boîte de dialogue msgbox

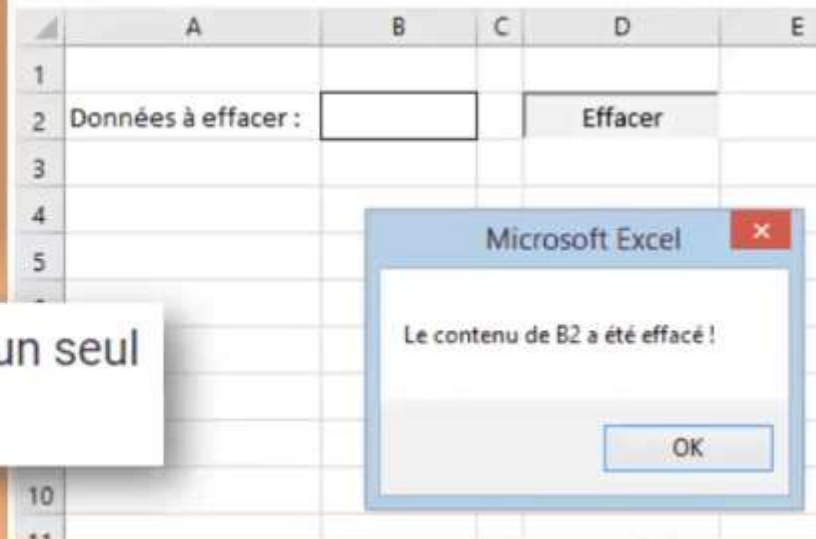
MSGBOX

Pour le moment, nous n'avons utilisé la boîte de dialogue `MsgBox` que pour afficher une information :

```
Sub effacerB2()  
  
    Range("B2").ClearContents  
    MsgBox "Le contenu de B2 a été effacé !"  
  
End Sub
```

Dans ce cas, `MsgBox` n'est utilisé qu'avec un seul argument.

Aperçu du résultat de ce code :



Utilisation des boîtes de dialogue prédéfinies

La boîte de dialogue messagebox

MSGBOX

Voici les 3 arguments que nous allons renseigner :

```
MsgBox([TEXTE], [BOUTONS], [TITRE])
```

- **Texte** : texte de la boîte de dialogue
- **Boutons** : choix des boutons (Oui, Non, Annuler, etc.) + d'autres options
- **Titre** : titre de la boîte de dialogue

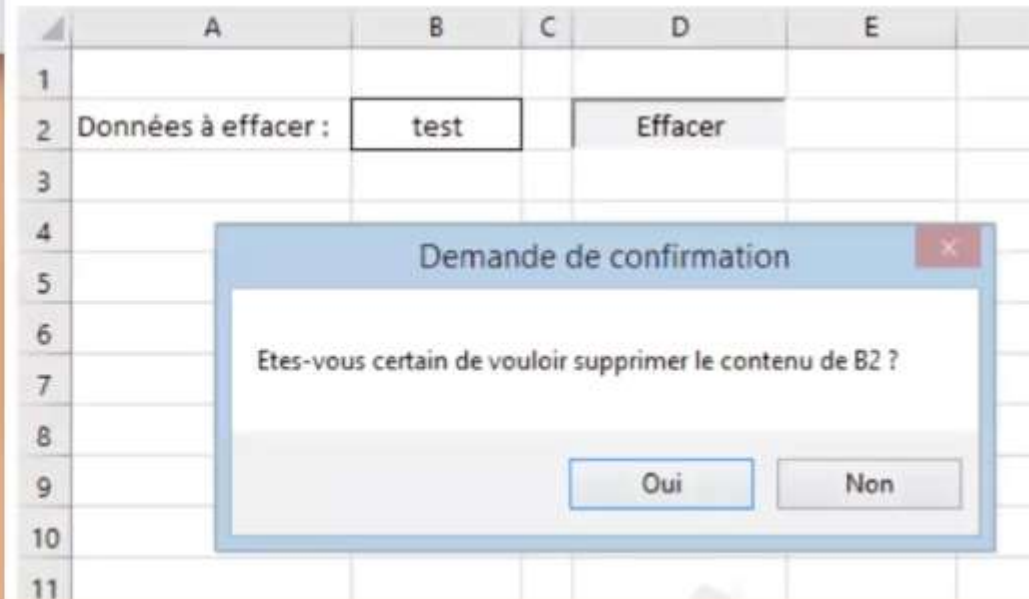
Utilisation des boîtes de dialogue prédéfinies

La boîte de dialogue msgbox

```
Sub effacerB2()  
  
    If MsgBox("Etes-vous certain de vouloir supprimer le contenu de B2 ?", vbYesNo, "Demande  
de confirmation") = vbYes Then  
        Range("B2").ClearContents  
        MsgBox "Le contenu de B2 a été effacé !"  
    End If  
  
End Sub
```

MSGBOX

Aperçu :



Utilisation de variables avec les messages

La boîte de dialogue msgbox

MSGBOX

`vbYesNo` indique que les boutons de la boîte de dialogue sont les boutons "Oui" et "Non", `vbYes` correspond au bouton "Oui" :

```
If MsgBox("Texte", vbYesNo, "Titre") = vbYes Then 'Si le bouton Oui est cliqué
```

Utilisation de variables avec les messages

La boîte de dialogue msgbox

MSGBOX

LES DIFFÉRENTES POSSIBILITÉS POUR LE SECOND ARGUMENT DE MSGBOX

Constante	Valeur	Description
vbOKOnly	0	OK
vbOKCancel	1	OK Annuler
vbAbortRetryIgnore	2	Abandonner Recommencer Ignorer
vbYesNoCancel	3	Oui Non Annuler
vbYesNo	4	Oui Non
vbRetryCancel	5	Recommencer Annuler

Utilisation de variables avec les messages

La boîte de dialogue msgbox

MSGBOX

vbCritical

16



vbQuestion

32



vbExclamation

48



vbInformation

64



Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

vbDefaultButton1	0	Bouton par défaut : Bouton 1
vbDefaultButton2	256	Bouton par défaut : Bouton 2
vbDefaultButton3	512	Bouton par défaut : Bouton 3
vbApplicationModal	0	Force l'utilisateur à répondre avant de poursuivre avec Excel
vbSystemModal	4096	Force l'utilisateur à répondre avant de poursuivre avec d'autres applications (boîte de dialogue au premier plan)

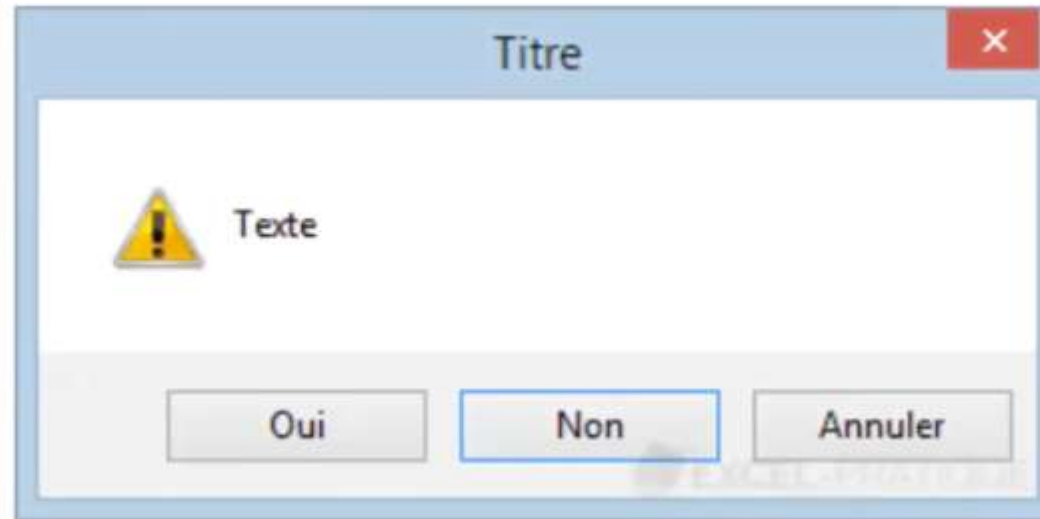
Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

```
MsgBox("Texte", vbYesNoCancel + vbExclamation + vbDefaultButton2, "Titre")
```

Aperçu :



Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

Les constantes peuvent être remplacées par leur valeur respective, ces 3 lignes affichent une boîte de dialogue identique :

```
MsgBox("Texte", vbYesNoCancel + vbExclamation + vbDefaultButton2, "Titre")  
MsgBox("Texte", 3 + 48 + 256, "Titre")  
MsgBox("Texte", 307, "Titre")
```

Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

LES VALEURS RENVOYÉES PAR MSGBOX

Constante	Valeur	Bouton correspondant à la valeur
vbOK	1	OK
vbCancel	2	Annuler
vbAbort	3	Abandonner
vbRetry	4	Recommencer
vbIgnore	5	Ignorer
vbYes	6	Oui
vbNo	7	Non

Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

Voici l'exemple d'une `MsgBox` qui apparaît en boucle tant que le bouton "Oui" n'est pas cliqué :

```
Sub humour()  
  
    Do  
        If MsgBox("Aimez-vous le site Excel-Pratique ?", 36, "Sondage") = vbYes Then  
            Exit Do 'Si réponse = Oui on sort de la boucle  
        End If  
    Loop While True 'Boucle infinie  
  
    MsgBox ";-)"  
  
End Sub
```

Personnalisation des boutons dans MsgBox

La boîte de dialogue msgbox

MSGBOX

SAUT DE LIGNE DANS UNE MSGBOX

```
MsgBox "Exemple 1" & Chr(10) & "Exemple 2" & Chr(10) & Chr(10) & "Exemple 3"
```

Aperçu :



Utilisation de l'InputBox

la boîte de dialogue inputbox

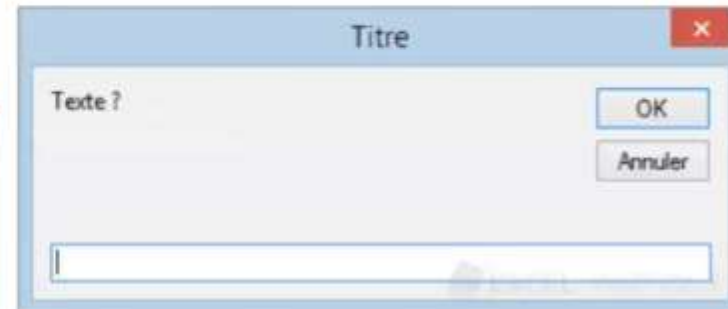
INPUTBOX

INPUTBOX

La fonction `InputBox` demande à l'utilisateur d'entrer une valeur dans une boîte de dialogue, exemple :

```
Sub exemple()  
  
    Dim resultat As String  
  
    resultat = InputBox("Texte ?", "Titre") 'La variable reçoit la valeur entrée dans  
l'InputBox  
  
    If resultat <> "" Then 'Si la valeur est différente de "" on affiche le résultat  
        MsgBox resultat  
    End If  
  
End Sub
```

Aperçu :



Utilisation de l'InputBox

la boîte de dialogue inputbox

INPUTBOX

Il est également possible d'indiquer une valeur par défaut en troisième argument :

```
InputBox("Texte ?", "Titre", "Valeur par défaut")
```

Aperçu :



Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_SHEETACTIVATE (À L'ACTIVATION D'UNE FEUILLE)

Cet événement se déclenche à chaque changement de feuille :

```
Private Sub Workbook_SheetActivate(ByVal Sh As Object)

End Sub
```

Par exemple, affichage du nom de la feuille dans une boîte de dialogue :

```
Private Sub Workbook_SheetActivate(ByVal Sh As Object)

    MsgBox "Nom de la feuille : " & Sh.Name

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_SHEETBEFOREDOUBLECLICK (AVANT DOUBLE-CLIC)

Cet événement se déclenche juste avant un double-clic sur une cellule :

```
Private Sub Workbook_SheetBeforeDoubleClick(ByVal Sh As Object, ByVal Target As Range, Cancel  
As Boolean)  
  
End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

Par exemple, coloration d'une cellule double-cliquée en fonction de la feuille :

```
Private Sub Workbook_SheetBeforeDoubleClick(ByVal Sh As Object, ByVal Target As Range, Cancel As Boolean)

    If Sh.Name = "Feuil1" Then
        Target.Interior.Color = RGB(255, 108, 0) 'Couleur orange
    Else
        Target.Interior.Color = RGB(136, 255, 0) 'Couleur verte
    End If

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_SHEETBEFORERIGHTCLICK (AVANT CLIC DROIT)

Cet événement se déclenche juste avant un clic droit sur une cellule :

```
Private Sub Workbook_SheetBeforeRightClick(ByVal Sh As Object, ByVal Target As Range, Cancel  
As Boolean)
```

```
End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_SHEETCHANGE (À CHAQUE MODIFICATION DE CELLULE)

Cet événement se déclenche à chaque modification du contenu d'une cellule :

```
Private Sub Workbook_SheetChange(ByVal Sh As Object, ByVal Target As Range)

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_SHEETSELECTIONCHANGE (À CHAQUE CHANGEMENT DE SÉLECTION)

Cet événement se déclenche à chaque changement de sélection sur une feuille de calcul :

```
Private Sub Workbook_SheetSelectionChange(ByVal Sh As Object, ByVal Target As Range)

End Sub
```


Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

Par exemple, coloration de la sélection si A1 est vide :

```
Private Sub Workbook_SheetSelectionChange(ByVal Sh As Object, ByVal Target As Range)

    If Range("A1") = "" Then
        Target.Interior.Color = RGB(124, 255, 255) 'Bleu clair
    End If

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKBOOK_NEWSSHEET (À L'INSERTION D'UNE FEUILLE)

Cet événement se déclenche à chaque insertion d'une nouvelle feuille :

```
Private Sub Workbook_NewSheet(ByVal Sh As Object)  
  
End Sub
```

Gestion des événements

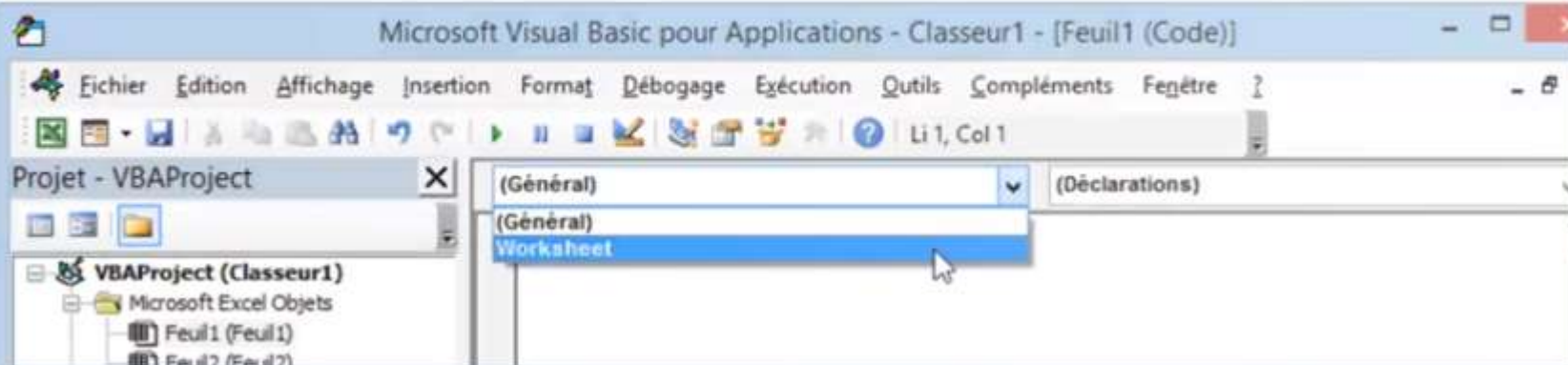
les evenements de feuilles

LES ÉVÉNEMENTS

les événements Worksheet

WORKSHEET_SELECTIONCHANGE (AU CHANGEMENT DE SÉLECTION)

Pour exécuter des instructions en fonction d'un événement pour une feuille en particulier, sélectionnez la feuille dans l'éditeur, puis **Worksheet** :



Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

les événements Worksheet

L'événement `SelectionChange` est ajouté par défaut, il agit lors d'un changement de sélection :

```
Private Sub Worksheet_SelectionChange(ByVal Target As Range)

End Sub
```

Gestion des événements

les evenements de feuilles

Par exemple, voici un code qui colore la ou les cellules sélectionnées et qui supprime automatiquement la coloration de la dernière sélection lors d'un changement de sélection :

```
Private Sub Worksheet_SelectionChange(ByVal Target As Range)

    Static selectionPrecedente As String

    'Suppression de la couleur de fond de la sélection précédente
    If selectionPrecedente <> "" Then
        Range(selectionPrecedente).Interior.ColorIndex = xlColorIndexNone
    End If

    'Coloration de la sélection actuelle
    Target.Interior.Color = RGB(181, 244, 0)

    'Enregistrement de l'adresse de la sélection actuelle
    selectionPrecedente = Target.Address

End Sub
```

Gestion des événements

les evenements de feuilles

WORKSHEET_ACTIVATE (À L'ACTIVATION DE LA FEUILLE)

Cet événement se déclenche lorsque la feuille est activée :

```
Private Sub Worksheet_Activate()  
  
End Sub
```

Par exemple, sélection de la cellule D5 à l'activation de la feuille :

```
Private Sub Worksheet_Activate()  
  
    Range("D5").Select  
  
End Sub
```


Gestion des événements

les evenements de feuilles

WORKSHEET_DEACTIVATE (À LA SORTIE DE LA FEUILLE)

Cet événement se déclenche lorsqu'une autre feuille du classeur est activée :

```
Private Sub Worksheet_Deactivate()  
  
End Sub
```

Par exemple, effacement du contenu des cellules B2 à B10 après avoir quitté la feuille :

```
Private Sub Worksheet_Deactivate()  
  
    Range("B2:B10").ClearContents  
  
End Sub
```


Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKSHEET_BEFOREDOUBLECLICK (AU DOUBLE-CLIC)

Cet événement se déclenche lors d'un double-clic sur une cellule de la feuille :

```
Private Sub Worksheet_BeforeDoubleClick(ByVal Target As Range, Cancel As Boolean)  
  
End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

Par exemple, coloration de la cellule double-cliquée en vert (ou en blanc si elle est déjà colorée) :

```
Private Sub Worksheet_BeforeDoubleClick(ByVal Target As Range, Cancel As Boolean)

    If Target.Interior.Color = 16777215 Then 'Si blanc
        Target.Interior.Color = RGB(200, 255, 100) 'Couleur verte
    Else 'Sinon
        Target.Interior.Color = 16777215 'Couleur blanche
    End If

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKSHEET_BEFORERIGHTCLICK (AU CLIC DROIT)

Cet événement se déclenche lors d'un clic droit sur la feuille :

```
Private Sub Worksheet_BeforeRightClick(ByVal Target As Range, Cancel As Boolean)  
  
End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

Par exemple, ajout de la **date du jour** par clic droit si la cellule cliquée est dans la colonne C :

```
Private Sub Worksheet_BeforeRightClick(ByVal Target As Range, Cancel As Boolean)

    'Si cellule de la colonne 3 (C)
    If Target.Column = 3 Then
        Target = Date 'Ajout de la date du jour
        Cancel = True 'Annulation du clic droit
    End If

End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

WORKSHEET_CHANGE (À CHAQUE MODIFICATION DE CELLULE)

Cet événement se déclenche lors de modifications du contenu des cellules de la feuille :

```
Private Sub Worksheet_Change(ByVal Target As Range)
```

```
End Sub
```

Gestion des événements

les evenements de feuilles

LES ÉVÉNEMENTS

DÉSACTIVER TEMPORAIREMENT TOUS LES ÉVÉNEMENTS

Pour exécuter du code sans déclencher d'événements, placez-le entre ces deux lignes :

```
Application.EnableEvents = False 'Désactive les événements
```

```
'Instructions ...
```

```
Application.EnableEvents = True 'Réactive les événements
```


Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

les événements Workbook

Jusque-là, nous avons lancé nos macros en cliquant sur un bouton.

Il est également possible de les exécuter automatiquement lors d'un événement particulier du classeur, tel que l'ouverture du classeur, sa fermeture, son enregistrement, etc.

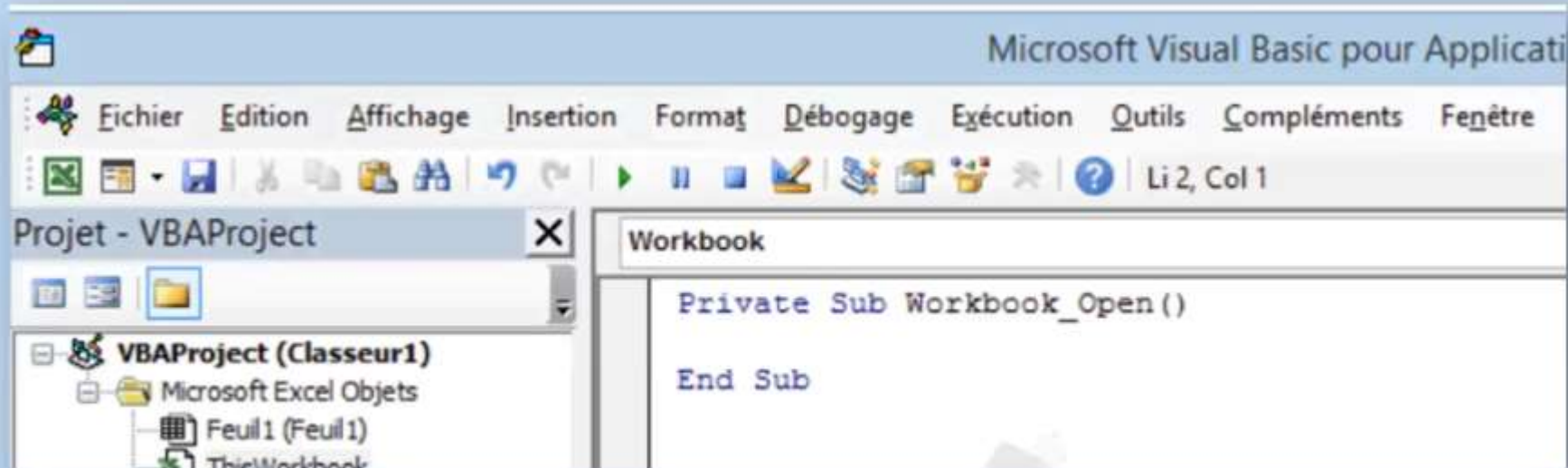
Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_OPEN (À L'OUVERTURE)

Pour exécuter des instructions à l'ouverture du classeur (après l'activation des macros par l'utilisateur), rendez-vous dans `ThisWorkbook` et sélectionnez `Workbook` :



Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_OPEN (À L'OUVERTURE)

Pour exécuter des instructions à l'ouverture du classeur (après l'activation des macros par l'utilisateur), rendez-vous dans `ThisWorkbook` et sélectionnez `Workbook` :

```
Private Sub Workbook_Open()  
  
    MsgBox "Message de bienvenue"  
  
End Sub
```

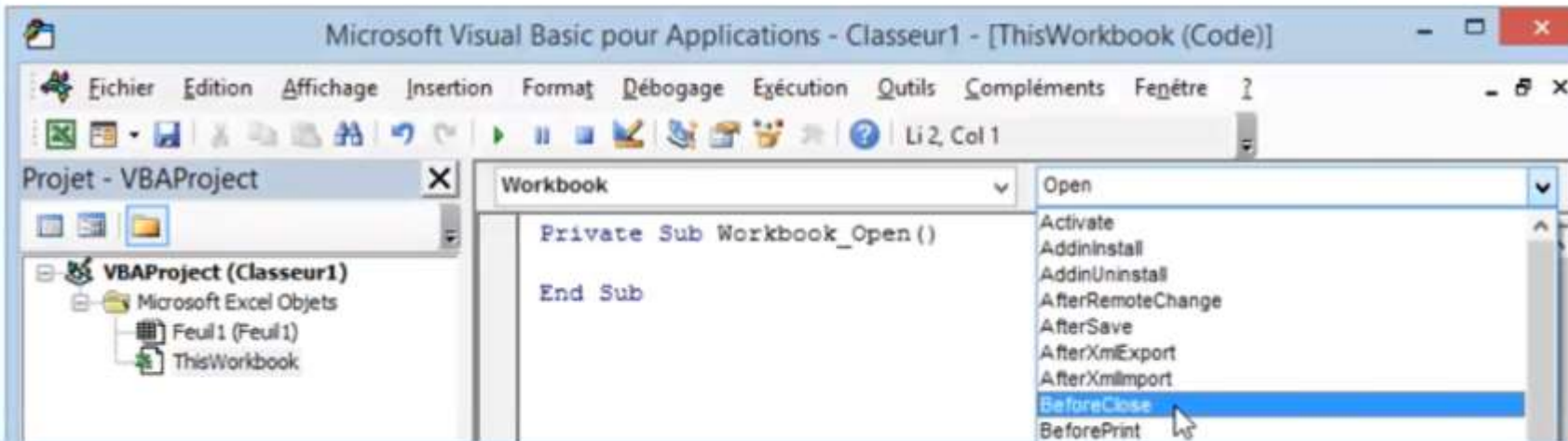
Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_BEFORECLOSE (AVANT FERMETURE)

Pour exécuter des instructions juste avant la fermeture du classeur, choisissez **BeforeClose** :



Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

```
Private Sub Workbook_BeforeClose(Cancel As Boolean)  
  
End Sub
```

Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

La fermeture du classeur peut être annulée en attribuant la valeur `True` à la variable `Cancel`.

Voici un exemple où l'utilisateur doit confirmer la fermeture du classeur :

```
Private Sub Workbook_BeforeClose(Cancel As Boolean)

    'Si l'utilisateur répond Non, la variable Cancel vaudra True (ce qui annulera la
    fermeture)
    If MsgBox("Etes-vous certain de vouloir fermer ce classeur ?", 36, "Confirmation") = vbNo
    Then
        Cancel = True
    End If

End Sub
```

Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_BEFORESAVE (AVANT ENREGISTREMENT)

Cet événement se déclenche juste avant l'enregistrement :

```
Private Sub Workbook_BeforeSave(ByVal SaveAsUI As Boolean, Cancel As Boolean)  
  
End Sub
```


Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_AFTERSAVE (APRÈS ENREGISTREMENT)

Cet événement se déclenche juste après l'enregistrement :

```
Private Sub Workbook_AfterSave(ByVal Success As Boolean)  
  
End Sub
```


Gestion des événements

les evenements du classeur

LES ÉVÉNEMENTS

WORKBOOK_BEFOREPRINT (AVANT IMPRESSION)

Cet événement se déclenche juste avant l'impression :

```
Private Sub Workbook_BeforePrint(Cancel As Boolean)  
  
End Sub
```

L'impression peut être annulée en attribuant la valeur `True` à la variable `Cancel` .

Création de procédures événementielles

Acces aux fichiers avec APPEND

```
Sub AjouterTexteSansSautDeLigne()  
    Dim cheminFichier As String  
    Dim numeroFichier As Integer  
    Dim texteAAjouter As String  
  
    ' Spécifiez le chemin du fichier  
    cheminFichier = "C:\chemin\vers\le\fichier.txt"  
    ' Obtenir un numéro de fichier disponible  
    numeroFichier = FreeFile  
    ' Ouvrir le fichier en mode Ajout (Append)  
    Open cheminFichier For Append As #numeroFichier  
    ' Texte à ajouter au fichier  
    texteAAjouter = "Texte ajouté sans saut de ligne."  
    ' Ecrire dans le fichier sans ajouter de retour à la ligne  
    ' Le point-virgule à la fin empêche le saut de ligne automatique  
    Print #numeroFichier, texteAAjouter;  
    ' Fermer le fichier  
    Close #numeroFichier  
    MsgBox "Texte ajouté avec succès sans saut de ligne!"  
End Sub
```

Création de procédures événementielles

Acces aux fichiers avec INPUT

```
Sub LireFichierAvecInput()  
    Dim cheminFichier As String  
    Dim numeroFichier As Integer  
    Dim ligneDeTexte As String  
    ' Spécifiez le chemin du fichier à lire  
    cheminFichier = "C:\chemin\vers\le\fichier.txt"  
    ' Obtenir un numéro de fichier disponible  
    numeroFichier = FreeFile  
    ' Ouvrir le fichier en mode Lecture (Input)  
    Open cheminFichier For Input As #numeroFichier  
    ' Lire le fichier ligne par ligne  
    Do While Not EOF(numeroFichier)  
        Line Input #numeroFichier, ligneDeTexte  
        ' Traiter la ligne lue (par exemple, afficher dans la fenêtre immédiate)  
        Debug.Print ligneDeTexte  
    Loop  
    ' Fermer le fichier après la lecture  
    Close #numeroFichier  
    MsgBox "Lecture du fichier terminée!"  
End Sub
```

Création de procédures événementielles

Acces aux fichiers avec OUTPUT

```
Sub EcrireFichierAvecOutput()  
    Dim cheminFichier As String  
    Dim numeroFichier As Integer  
    Dim texteAEcrire As String  
    ' Spécifiez le chemin du fichier à créer ou écraser  
    cheminFichier = "C:\chemin\vers\le\fichier.txt"  
    ' Obtenir un numéro de fichier disponible  
    numeroFichier = FreeFile  
    ' Ouvrir le fichier en mode Ecriture (Output)  
    Open cheminFichier For Output As #numeroFichier  
    ' Texte à écrire dans le fichier  
    texteAEcrire = "Ceci est le texte écrit dans le fichier." & vbCrLf  
    texteAEcrire = texteAEcrire & "Cette opération écrase tout contenu existant."  
    ' Ecrire dans le fichier  
    Print #numeroFichier, texteAEcrire  
    ' Fermer le fichier après l'écriture  
    Close #numeroFichier  
    MsgBox "Ecriture dans le fichier terminée!"  
End Sub
```


Création de procédures événementielles

la macro autoOpen

```
Private Sub Workbook_Open()  
    Sheets(1).Activate  
    MsgBox "Bienvenue dans ce classeur Excel!"  
End Sub
```

Gestion des erreurs et débogage

Le debogage



```
Private Sub Workbook_Open()
```

```
    Sheets(1).Activate
```

```
    MsgBox "Bienvenue dans ce classeur Excel!"
```

```
End Sub
```

Prévention des erreurs

Le mot clef GOTO

```
Sub ExempleGestionErreur()  
  On Error GoTo GestionErreur
```

```
  ' Essayons de diviser par zéro (provoque une erreur)
```

```
  Dim resultat As Double  
  resultat = 10 / 0
```

```
  MsgBox "Cette ligne ne sera pas exécutée en cas d'erreur."
```

```
  Exit Sub ' Sortie de la procédure pour éviter d'exécuter le code d'erreur
```

```
GestionErreur:
```

```
  MsgBox "Une erreur s'est produite : " & Err.Description
```

```
End Sub
```


Prévention des erreurs

Le mot clef ON ERROR

```
Sub ExempleGestionErreur()  
    On Error GoTo GestionErreur  
    ' Essayons de diviser par zéro (provoque une erreur)  
    Dim resultat As Double  
    resultat = 10 / 0  
    MsgBox "Cette ligne ne sera pas exécutée en cas d'erreur."  
    Exit Sub ' Sortie de la procédure pour éviter d'exécuter le code d'erreur  
GestionErreur:  
    MsgBox "Une erreur s'est produite : " & Err.Description  
End Sub
```

```
Sub ExempleOnErrorResumeNext()  
    On Error Resume Next ' Ignore les erreurs et continue  
    ' Code susceptible de générer une erreur  
    Dim resultat As Double  
    resultat = 10 / 0 ' Cette ligne provoquera une erreur, mais sera ignorée  
    MsgBox "Le programme continue, même si une erreur s'est produite."  
    On Error GoTo 0 ' Réinitialise la gestion des erreurs (désactive On Error Resume Next)  
End Sub
```

Prévention des erreurs

Les différents mots clef EXIT

```
Sub ExempleExitSub()  
    MsgBox "Avant Exit Sub"  
  
    ' Condition pour sortir de la procédure  
    If True Then  
        Exit Sub  
    End If  
  
    MsgBox "Après Exit Sub" ' Cette ligne ne sera pas exécutée  
End Sub
```

```
Function CalculerSomme(x As Double, y As Double) As Double  
    ' Condition pour sortir de la fonction  
    If x < 0 Or y < 0 Then  
        MsgBox "Les valeurs doivent être positives."  
        Exit Function  
    End If  
  
    ' Calculer et retourner la somme  
    CalculerSomme = x + y  
End Function
```

Prévention des erreurs

Les erreurs de compilation

```
Sub Exemple ExitSub()
```

```
    MsgBox "Avant Exit Sub"
```

```
    ' Condition pour sortir de la procédure
```

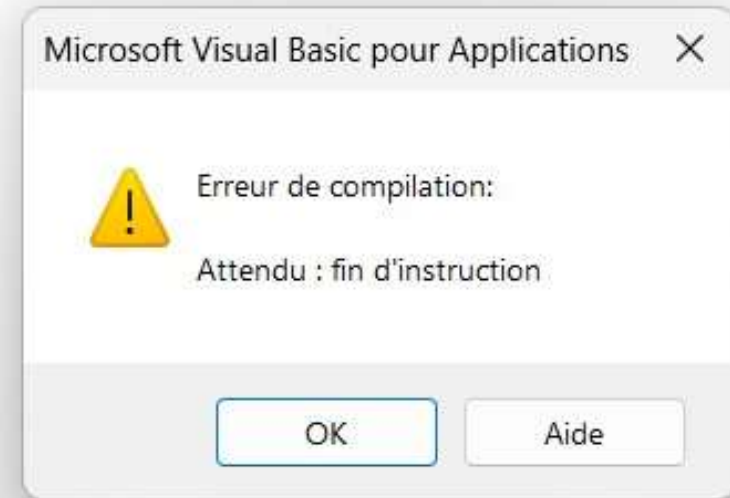
```
    If True Then
```

```
        Exit Sub
```

```
    End If
```

```
    MsgBox "Après Exit Sub" ' Cette ligne ne sera pas exécutée
```

```
End Sub
```

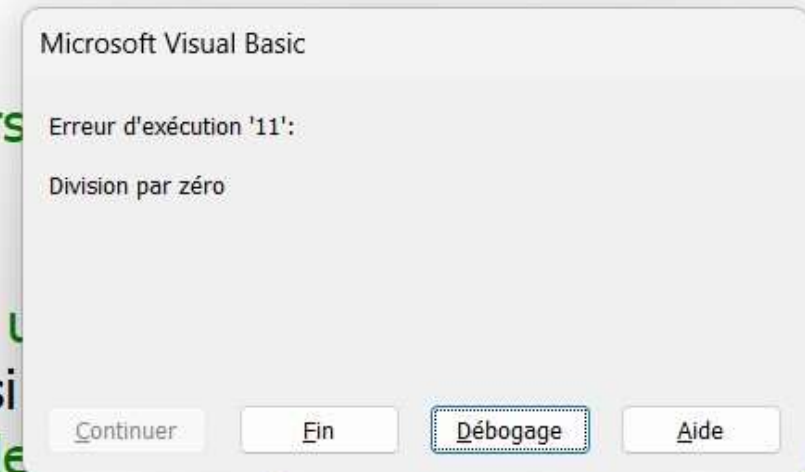


Prévention des erreurs

Les erreurs d'execution

Sub ExempleOnErrorResumeNext()

```
' On Error Resume Next ' Ignore les erreurs  
' Code susceptible de générer une erreur  
Dim resultat As Double  
resultat = 10 / 0 ' Cette ligne provoquera u  
MsgBox "Le programme continue, même si  
' On Error GoTo 0 ' Réinitialise la gestion de  
End Sub
```



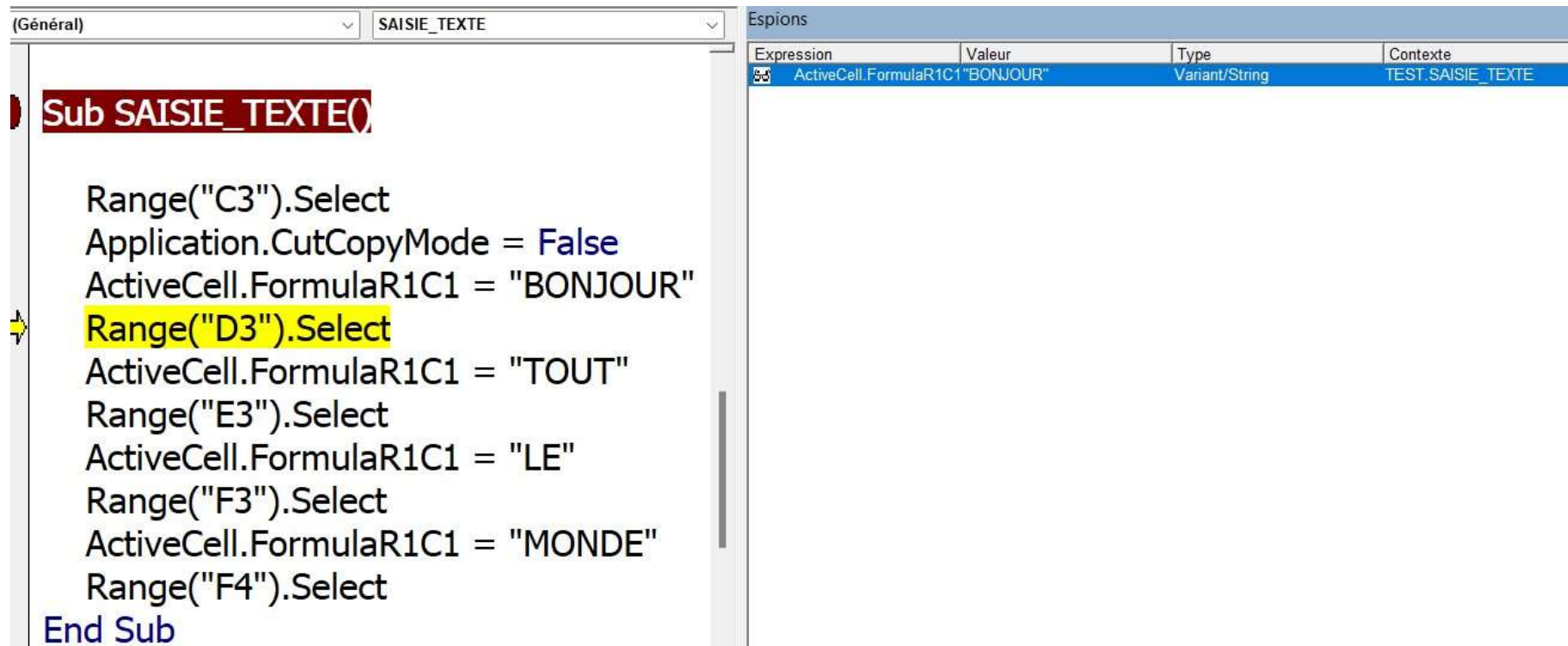
Prévention des erreurs

Redirection du code avec les etiquettes

```
Sub ExempleEtiquettesBoucle()  
    Dim i As Integer  
  
    For i = 1 To 10  
        ' Condition pour sortir de la boucle  
        If i = 5 Then  
            GoTo FinBoucle  
        End If  
  
        Debug.Print i  
    Next i  
  
FinBoucle:  
    ' Code à exécuter après la sortie de la boucle  
    MsgBox "La boucle est terminée ou interrompue à i = " & i  
End Sub
```


Utilisation des outils de débogage

la fenetre espion



The screenshot displays the Visual Basic Editor interface. On the left, the 'Général' (General) tab is selected, showing the code for the 'SAISIE_TEXTE' subroutine. The code includes several range selections and assignments to the 'ActiveCell.FormulaR1C1' property. The line 'Range("D3").Select' is highlighted in yellow. On the right, the 'Espions' (Spies) window is open, showing a table with the following data:

Expression	Valeur	Type	Contexte
ActiveCell.FormulaR1C1	"BONJOUR"	Variant/String	TEST.SAISIE_TEXTE

Utilisation des outils de débogage

Les points d'arrêt



```
Sub SAISIE_TEXTE()
```

```
    Range("C3").Select
```

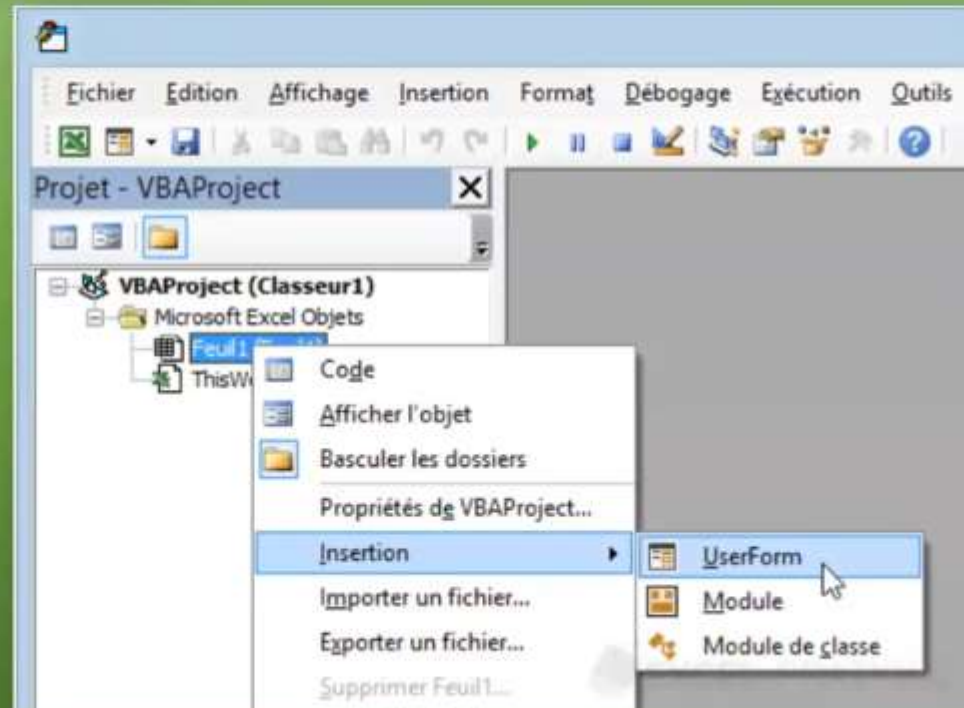
```
    Application.CutCopyMode = False
```

```
    ActiveCell.FormulaR1C1 = "BONJOUR"
```

```
    Range("D3").Select
```


les userform ou formulaires et les controles

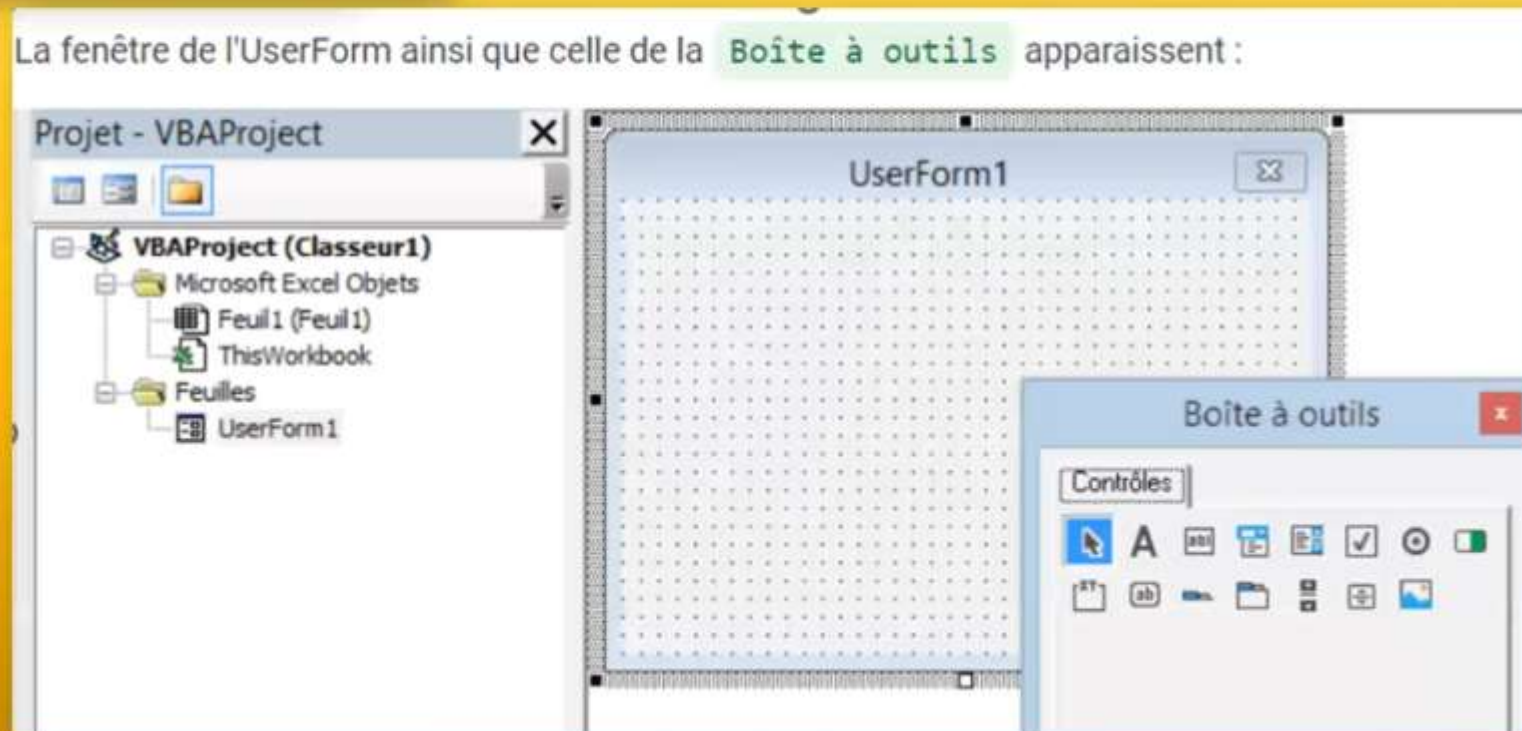
UserForm



les userform ou formulaires et les controles

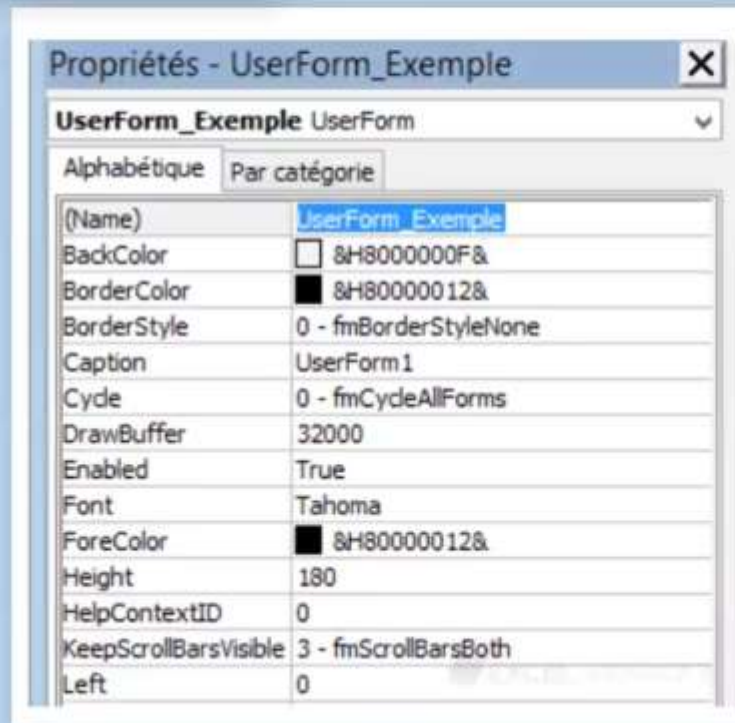
UserForm

La fenêtre de l'UserForm ainsi que celle de la Boîte à outils apparaissent :



les userform ou formulaires et les controles

UserForm

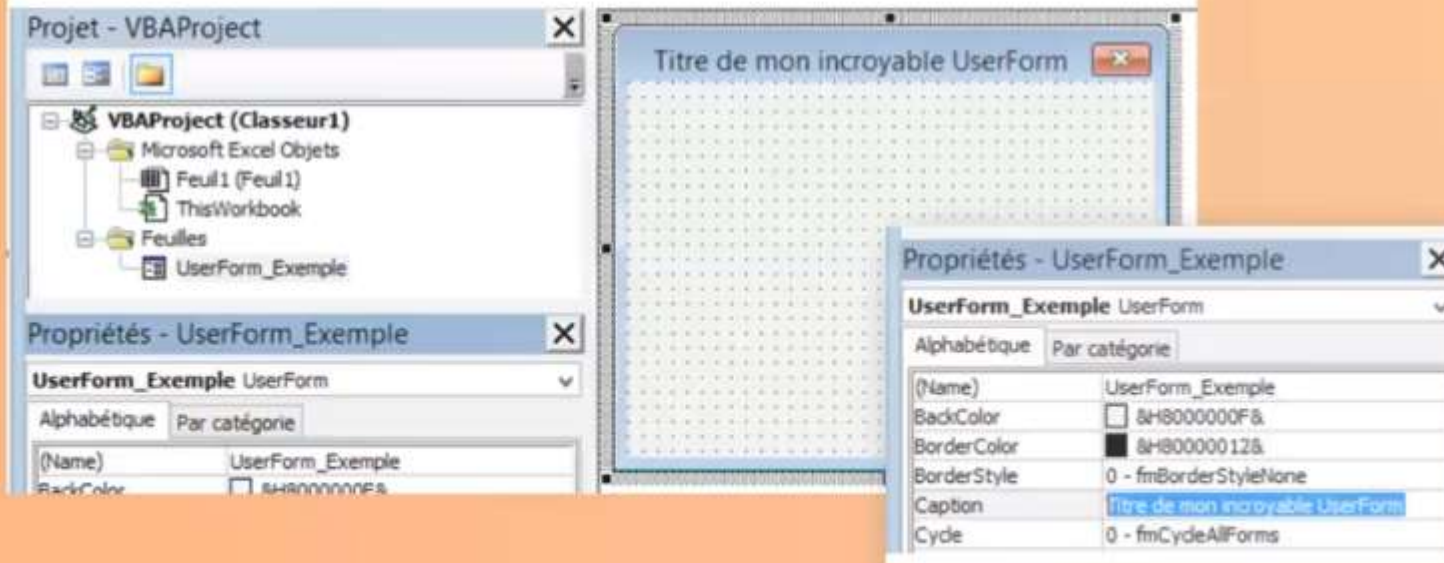


les userform ou formulaires et les controles

UserForm

TITRE DE L'USERFORM

Pour modifier le titre de l'UserForm, modifiez sa propriété **Caption** :

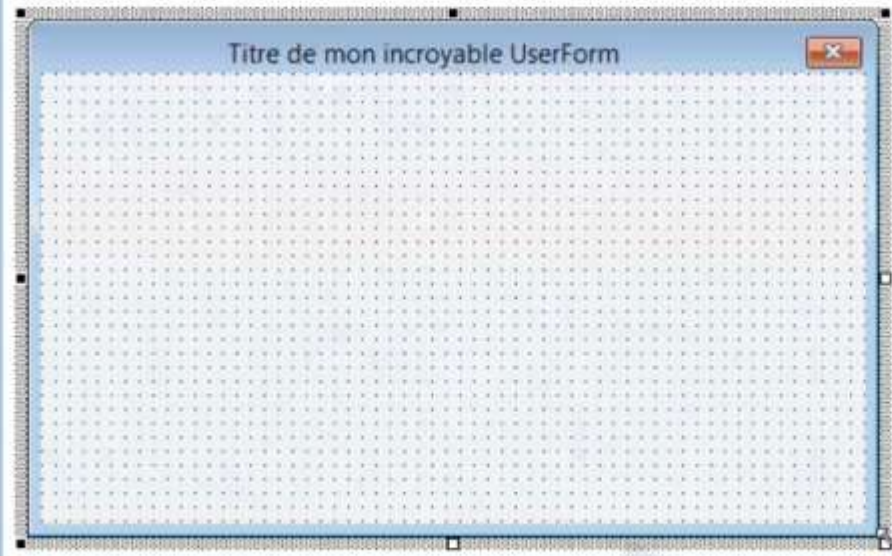


les userform ou formulaires et les controles

UserForm

DIMENSIONS DE L'USERFORM

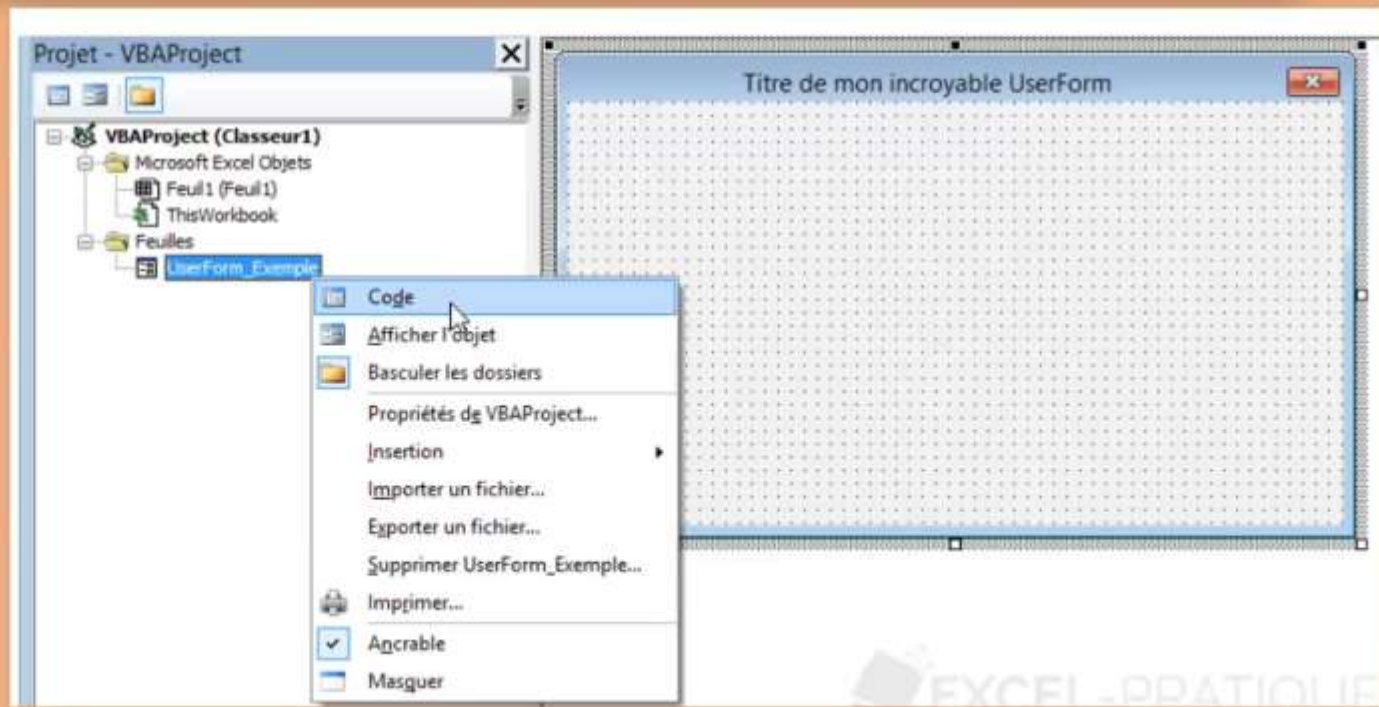
Pour modifier les dimensions de l'UserForm, modifiez ses propriétés `Width` et `Height` ou redimensionnez l'UserForm à la main :



les userform ou formulaires et les controles

UserForm

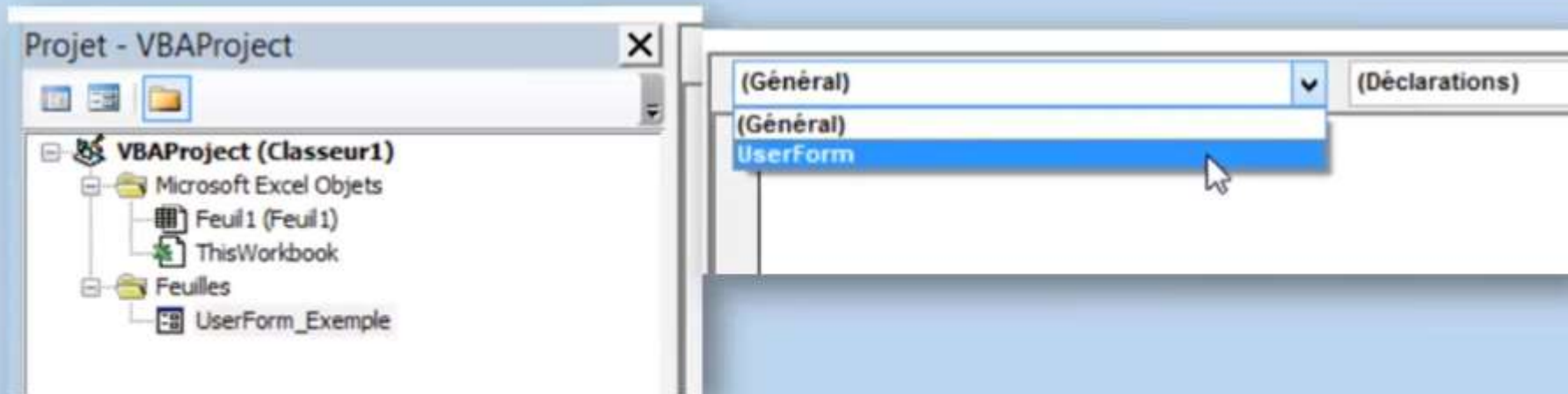
Les propriétés d'un UserForm peuvent également être modifiées à partir d'un code VBA.



les userform ou formulaires et les controles

UserForm

```
Private Sub UserForm_Initialize()  
  
End Sub
```



UserForm

```
Private Sub UserForm_Initialize()  
    UserForm_Exemple.  
End Sub
```



UserForm

La propriété **Height** est la hauteur et **Width** la largeur :

```
Private Sub UserForm_Initialize()  
  
    UserForm_Exemple.Height = 250  
    UserForm_Exemple.Width = 250  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

Pour simplifier le code, nous pouvons remplacer le nom de l'UserForm par **Me** (puisque ce code est placé dans l'UserForm sur lequel on souhaite agir) :

```
Private Sub UserForm_Initialize()
```

```
    Me.Height = 250
```

```
    Me.Width = 250
```

```
End Sub
```

```
Private Sub UserForm_Click()
```

```
    Me.Height = Me.Height + 50
```

```
    Me.Width = Me.Width + 50
```

```
End Sub
```

UserForm

LANCER UN USERFORM

Pour lancer un UserForm à partir d'une procédure, utilisez `Show`

```
Sub lancerUserform()  
  
    UserForm_Exemple.Show  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

Les contrôles sont les éléments (boutons, intitulés, zone de texte, cases à cocher, etc.) qui peuvent être insérés sur un UserForm (ou sur une feuille Excel).

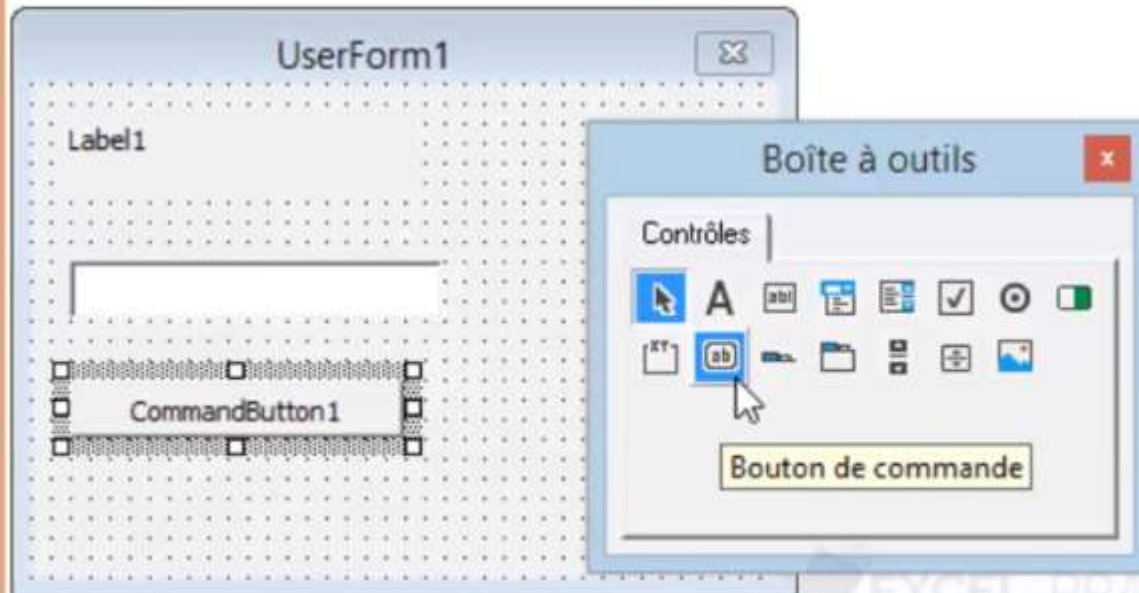
Les contrôles ont également toute une panoplie de propriétés et d'événements qui diffèrent d'un contrôle à l'autre.

les userform ou formulaires et les controles

UserForm

les contrôles

Pour commencer, ajoutez un UserForm et insérez les 3 contrôles suivants : un intitulé `Label1` , une zone de texte `TextBox` et un bouton `CommandButton` :

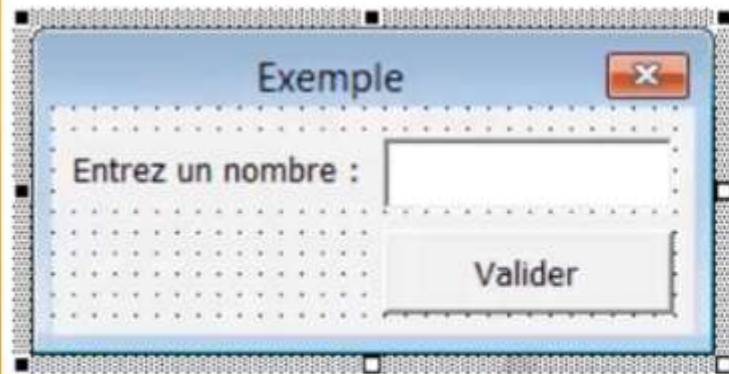


les userform ou formulaires et les controles

UserForm

les contrôles

Modifiez les propriétés de l'UserForm et des contrôles (dont les propriétés (Name) pour le nom, Caption pour le texte et Font pour la taille du texte) pour obtenir ceci :

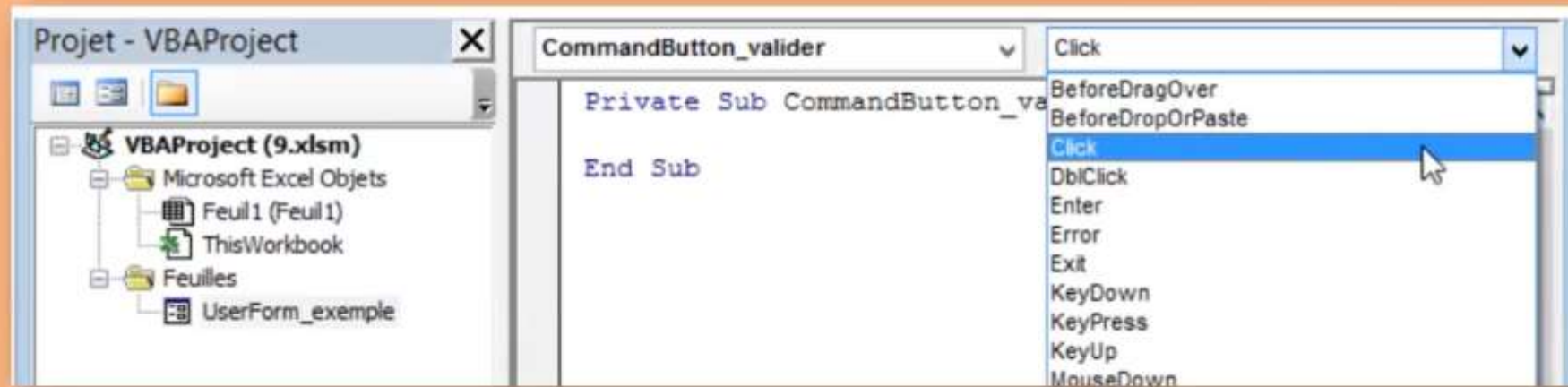


The screenshot shows a VBA UserForm titled "Exemple" with a standard Windows-style title bar (minimize, maximize, close buttons). The form contains a label "Entrez un nombre :" followed by a text input field. Below the input field is a button labeled "Valider". The form is set against a dotted grid background, which is typical for VBA UserForms.

les userform ou formulaires et les controles

UserForm

les contrôles



les userform ou formulaires et les controles

UserForm

les contrôles

L'événement `click` est déclenché au clic sur le bouton :

```
Private Sub CommandButton_valider_Click()  
  
    'La cellule A1 (de la feuille active) obtient la valeur de la zone de texte nommée  
    "TextBox_nombre"  
    Range("A1") = TextBox_nombre.Value  
  
    'Fermeture (Unload) de l'UserForm (Me)  
    Unload Me  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub TextBox_nombre_Change()  
  
    If IsNumeric(TextBox_nombre.Value) Then 'Si valeur numérique  
        TextBox_nombre.BackColor = RGB(255, 255, 255) 'Blanc  
    Else 'Sinon  
        TextBox_nombre.BackColor = RGB(247, 205, 201) 'Rouge clair  
    End If  
  
End Sub
```

Aperçu :

The screenshot shows a spreadsheet application with a modal dialog box titled "Exemple". The dialog has a text input field labeled "Entrez un nombre :" containing the letter "w". Below the input field is a button labeled "Valider". The spreadsheet background is visible, showing columns A through E and rows 1 through 9.

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub CommandButton_valider_Click()  
  
    'Si valeur numérique  
    If IsNumeric(TextBox_nombre.Value) Then  
        Range("A1") = TextBox_nombre.Value  
        Unload Me  
    End If  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

LES CASES À COCHER (CHECKBOX)

Voici un exemple d'utilisation de cases à cocher dont l'objectif est de modifier les valeurs en colonne B en fonction des cases cochées dans l'UserForm :

	A	B	C	D	E
1	Numéro	Choix			
2	1	Non			
3	2	Non			
4	3	Non			
5					
6					
7					
8					
9					
10					
11					
12					

Ouvrir l'UserForm

Exemple

Cochez votre sélection :

☐ Numéro 1

☐ Numéro 2

☐ Numéro 3

Valider

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub CommandButton_valider_Click()
```

```
    'Numéro 1
```

```
    If CheckBox1.Value = True Then 'Si coché
```

```
        Range("B2") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B2") = "Non"
```

```
    End If
```

```
    'Numéro 2
```

```
    If CheckBox2.Value = True Then 'Si coché
```

```
        Range("B3") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B3") = "Non"
```

```
    End If
```

```
    'Numéro 3
```

```
    If CheckBox3.Value = True Then 'Si coché
```

```
        Range("B4") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B4") = "Non"
```

```
    End If
```

```
    'Fermeture
```

```
    Unload Me
```

```
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

```
If CheckBox1.Value Then 'Si coché
```

L'ajout de `.Value` est donc facultatif :

```
If CheckBox1 Then 'Si coché
```

```
If CheckBox1.Value = True Then 'Si coché
```

```
If CheckBox1.Value Then 'Si coché
```


les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub CommandButton_valider_Click()
```

```
    'Numéro 1
```

```
    If CheckBox1 Then 'Si coché
```

```
        Range("B2") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B2") = "Non"
```

```
    End If
```

```
    'Numéro 2
```

```
    If CheckBox2 Then 'Si coché
```

```
        Range("B3") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B3") = "Non"
```

```
    End If
```

```
    'Numéro 3
```

```
    If CheckBox3 Then 'Si coché
```

```
        Range("B4") = "Oui"
```

```
    Else 'Si décoché
```

```
        Range("B4") = "Non"
```

```
    End If
```

```
    'Fermeture
```

```
    Unload Me
```

```
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

Maintenant, imaginez que vous n'ayez pas 3 mais 30 cases à cocher ...

Dans ce cas, l'utilisation d'une boucle est plus que bienvenue :

```
Private Sub CommandButton_valider_Click()  
  
    Dim i As Integer  
  
    'Boucle des cases à cocher  
    For i = 1 To 3  
        If Controls("CheckBox" & i) Then 'Si coché  
            Range("B" & i + 1) = "Oui"  
        Else 'Si décoché  
            Range("B" & i + 1) = "Non"  
        End If  
    Next  
  
    'Fermeture  
    Unload Me  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub UserForm_Initialize()  
  
    If Range("B2") = "Oui" Then  
        CheckBox1 = True  
    End If  
  
    If Range("B3") = "Oui" Then  
        CheckBox2 = True  
    End If  
  
    If Range("B4") = "Oui" Then  
        CheckBox3 = True  
    End If  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub UserForm_Initialize()  
  
    If Range("B2") = "Oui" Then CheckBox1 = True  
  
    If Range("B3") = "Oui" Then CheckBox2 = True  
  
    If Range("B4") = "Oui" Then CheckBox3 = True  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

L'utilisation d'une boucle est également possible :

```
Private Sub UserForm_Initialize()  
  
    Dim i As Integer  
  
    For i = 1 To 3  
        If Range("B" & i + 1) = "Oui" Then Controls("CheckBox" & i) = True  
    Next  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

	A	B	C	D	E
1	Numéro	Choix			
2	1	Oui			
3	2	Non			
4	3	Oui			
5					
6					
7					
8					
9					
10					
11					
12					

Ouvrir l'UserForm

Exemple

Cochez votre sélection :

☒ Numéro 1

☐ Numéro 2

☒ Numéro 3

Valider

les userform ou formulaires et les controles

UserForm

les contrôles

Sondage

Que pensez-vous de ce cours VBA ?

- ☐ Passionnant
- ☐ Intéressant
- ☐ Bof
- ☐ Une horreur

Prévoyez-vous de développer un premier projet en VBA à la fin de ce cours ?

- ☐ Oui
- ☐ Non
- ☐ Je ne sais pas
- ☐ Chaque chose en son temps

Enregistrer

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub CommandButton_valider_Click()

    Dim i As Integer

    'Question 1
    For i = 1 To 4
        If Controls("OptionButton_a_" & i) Then Range("A2") = Controls("OptionButton_a_" &
i).Caption
    Next

    'Question 2
    For i = 1 To 4
        If Controls("OptionButton_b_" & i) Then Range("B2") = Controls("OptionButton_b_" &
i).Caption
    Next

    'Fermeture
    Unload Me

End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

LA LISTE DÉROULANTE (COMBOBOX) ET LA ZONE DE LISTE (LISTBOX)

	A	B	C	D	E	F	G
1	Suisse	France	Belgique	Canada		Ouvrir l'UserForm	
2	Zürich	Paris	Bruxelles	Montréal			
3	Genève	Lyon	Anvers	Québec			
4	Lausanne	Toulouse	Namur				
5	Berne	Nantes	Bruges				
6	Sion	Lille					
7		Bordeaux					
8		Rennes					
9							
10							
11							
12							
13							
14							
15							
16							

Exemple

Pays :

Villes :

Valider

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub UserForm_Initialize()  
  
    Dim i As Integer  
  
    'Boucle pour ajouter les 4 pays à la liste déroulante  
    For i = 1 To 4  
        ComboBox_pays.AddItem Cells(1, i)  
    Next  
  
End Sub
```

The screenshot shows a VBA UserForm titled "Exemple". It contains two controls: a dropdown menu labeled "Pays :" and a list box labeled "Villes :". The dropdown menu is currently empty. The list box contains four items: "Suisse", "France", "Belgique", and "Canada". At the bottom of the form is a button labeled "Valider".

les userform ou formulaires et les controles

```
Private Sub ComboBox_Pays_Change()
```

```
    Dim colonne As Integer, nblignes As Integer
```

```
    'Zone de liste vidée (sinon les villes sont ajoutées à la suite)
```

```
    ListBox_villes.Clear
```

```
    'Numéro de la sélection
```

```
    colonne = ComboBox_pays.ListIndex + 1
```

```
    'Si le numéro de colonne = 0 (donc si aucun pays sélectionné) la procédure est quittée
```

```
    If colonne = 0 Then Exit Sub
```

```
    'Nombre de lignes de la colonne du pays choisi
```

```
    nblignes = Cells(1, colonne).End(xlDown).Row
```

```
    'Boucle pour ajouter les villes dans la zone de liste
```

```
    For i = 2 To nblignes
```

```
        ListBox_villes.AddItem Cells(i, colonne)
```

```
    Next
```

```
End Sub
```

UserForm

les contrôles

les userform ou formulaires et les controles

UserForm

les contrôles

```
Private Sub CommandButton_valider_Click()
```

```
    MsgBox "Ville sélectionnée : " & ListBox_villes '(propriété Value de ListBox_villes)
```

```
End Sub
```

	A	B	C	D	E	F	G	H	I
1	Suisse	France	Belgique	Canada		Ouvrir l'UserForm			
2	Zürich	Paris	Bruxelles	Montréal					
3	Genève	Lyon	Anvers	Québec					
4	Lausanne	Toulouse	Namur						
5	Berne	Nantes	Bruges						
6	Sion	Lille							
7		Bordeaux							
8		Rennes							
9									
10									
11									
12									
13									
14									
15									

Exemple

Pays : Suisse

Villes : Zürich
Genève
Lausanne
Berne
Sion

Valider

Microsoft Excel

Ville sélectionnée : Sion

OK

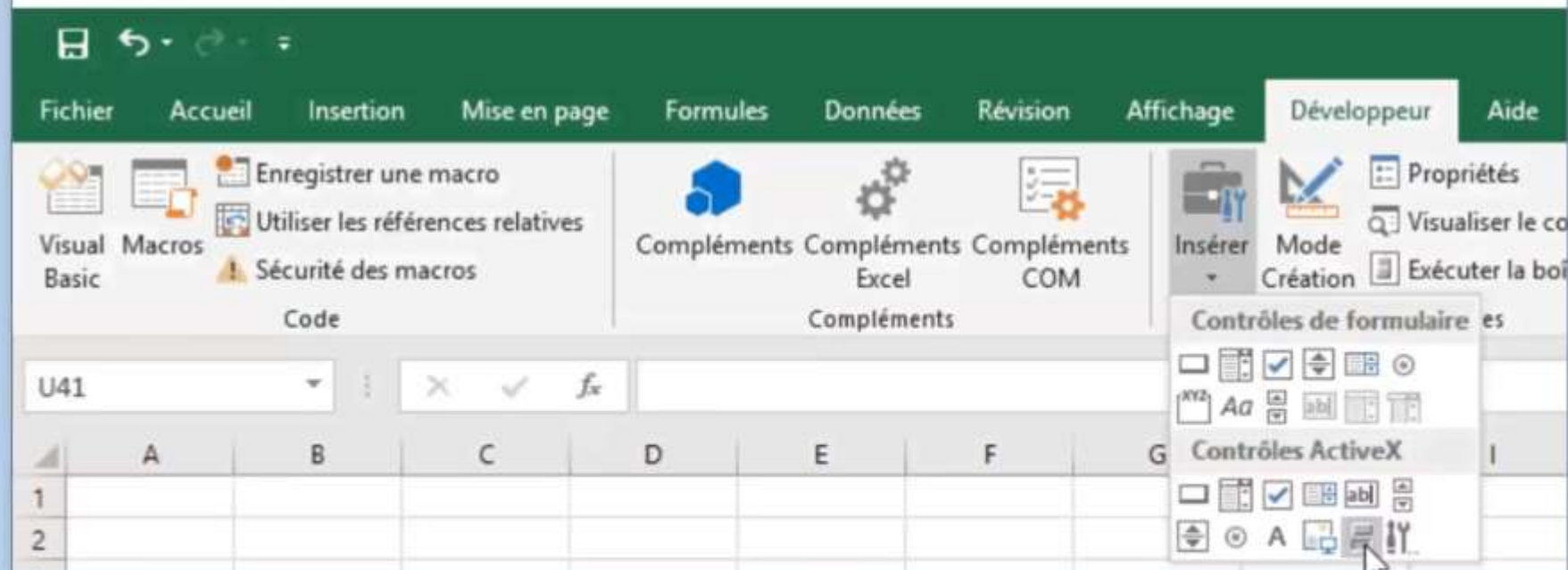
les userform ou formulaires et les controles

UserForm

les contrôles

LE BOUTON BASCULE (TOGGLEBUTTON)

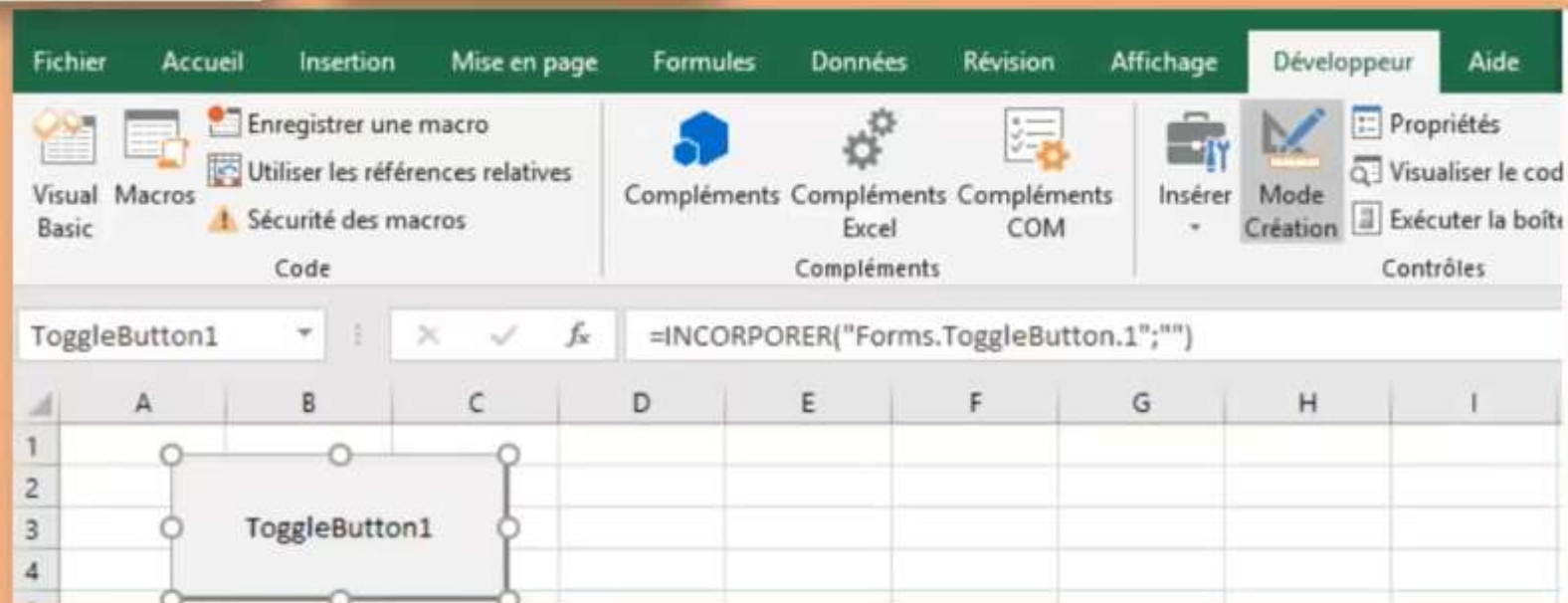
Insérez pour commencer un bouton bascule (contrôle ActiveX) à partir de l'onglet **Développeur** :



les userform ou formulaires et les controles

UserForm

les contrôles



Double-cliquez maintenant sur le bouton et modifiez ses propriétés pour obtenir :

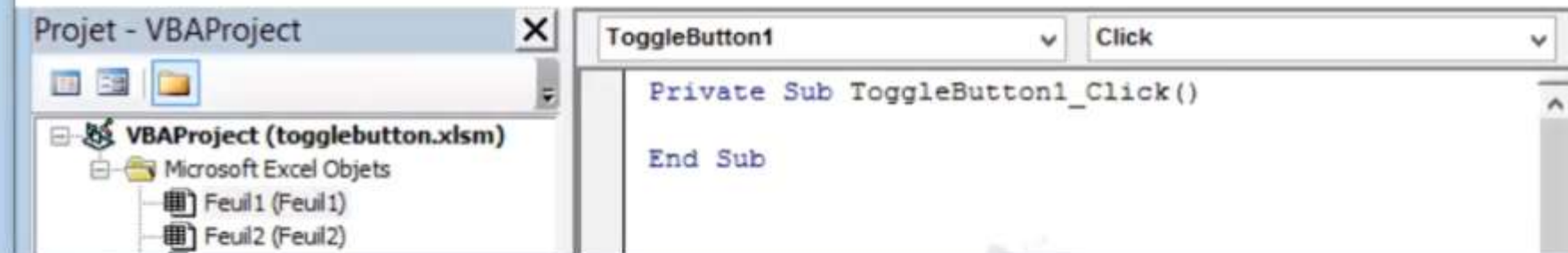
	A	B	C	D	E
1					
2		La feuille 2 est affichée			
3					
4					

les userform ou formulaires et les controles

UserForm

les contrôles

Le précédent double-clic a également ajouté l'événement **Click** du bouton dans la feuille où se trouve le bouton :



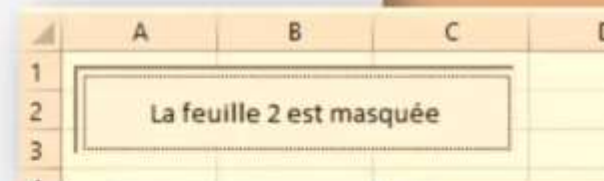
les userform ou formulaires et les controles

UserForm

les contrôles

Il ne reste plus qu'à entrer les instructions à exécuter au clic sur le bouton :

```
Private Sub ToggleButton1_Click()  
  
    'Si le bouton est pressé  
    If ToggleButton1 Then  
  
        'Masquer la feuille et modifier le texte du bouton  
        Sheets("Feuil2").Visible = 2  
        ToggleButton1.Caption = "La feuille 2 est masquée"  
  
    'Sinon  
    Else  
  
        'Afficher la feuille et modifier le texte du bouton  
        Sheets("Feuil2").Visible = -1  
        ToggleButton1.Caption = "La feuille 2 est affichée"  
  
    End If  
  
End Sub
```



A screenshot of an Excel spreadsheet with columns A, B, C, and D, and rows 1, 2, and 3. A message box is displayed over the spreadsheet, containing the text "La feuille 2 est masquée".

	A	B	C	D
1				
2				
3				

les userform ou formulaires et les controles

UserForm

les contrôles

The screenshot shows an Excel spreadsheet with columns labeled A through H. Columns A through F are used for a contact list with headers: A (Civilité), B (Nom), C (Prénom), D (Adresse), E (Lieu), and F (Pays). A button labeled 'Ajouter un contact' is located in cell G4. Overlaid on the spreadsheet is a UserForm dialog box titled 'Ajouter un contact'. The form contains the following controls: a Civilité section with radio buttons for 'Mme', 'Mlle', and 'M'; text input fields for 'Nom', 'Prénom', and 'Adresse'; a 'Lieu' text input field; and a 'Pays' dropdown menu. At the bottom of the form are two buttons: 'Fermer' and 'Ajouter'.

EXERCICE PRATIQUE

Vous l'aurez compris, l'objectif est de remplir le tableau de la feuille à l'aide du formulaire.

Quelques points à prendre en compte :

- Lister les pays en fonction de la liste de la seconde feuille
- Vérifier si tous les champs ont été renseignés avant d'ajouter un nouveau contact
- Après ajout d'un contact, réinitialiser le formulaire sans le fermer

Prenez un moment pour réaliser cet exercice avant de passer à la solution ...

les userform ou formulaires et les controles

UserForm

les contrôles

EXERCICE PRATIQUE

solution

LE BOUTON "FERMER"

Événement **Click** du bouton **Fermer** :

```
Private Sub CommandButton_fermer_Click()
```

```
    Unload Me
```

```
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

EXERCICE PRATIQUE

solution

LE CONTENU DE LA LISTE DÉROULANTE

Événement `Initialize` de l'UserForm :

```
Private Sub UserForm_Initialize()  
  
    Dim i As Integer  
  
    'Boucle pour ajouter les pays dans la liste déroulante  
    For i = 1 To 231  
        ComboBox_pays.AddItem Sheets("Pays").Cells(i, 1)  
    Next  
  
End Sub
```

les userform ou formulaires et les controles

UserForm

les contrôles

EXERCICE PRATIQUE

solution

```
Private Sub CommandButton_ajouter_Click()  
  
    'Si formulaire complet  
    If (OptionButton_1 Or OptionButton_2 Or OptionButton_3) And TextBox_nom <> "" And  
    TextBox_prenom <> "" And TextBox_adresse <> "" And TextBox_lieu <> "" And  
    ComboBox_pays.ListIndex >= 0 Then  
  
        'Instructions pour insérer le contact ici ...  
  
    'Si formulaire incomplet  
    Else  
        MsgBox "Formulaire incomplet"  
    End If  
  
End Sub
```

VÉRIFICATION DES CONTRÔLES

les userform ou formulaires et les controles

UserForm

les contrôles

EXERCICE PRATIQUE

solution

```
Private Sub CommandButton_ajouter_Click()  
  
    'Coloration des Labels en noir (&H80000012 = couleur de base de la propriété ForeColor)  
    Label_civilite.ForeColor = &H80000012  
    Label_nom.ForeColor = &H80000012  
    Label_prenom.ForeColor = &H80000012  
    Label_adresse.ForeColor = &H80000012  
    Label_lieu.ForeColor = &H80000012  
    Label_pays.ForeColor = &H80000012
```


les userform ou formulaires et les controles

```
'Contrôles des champs
If OptionButton_1 = False And OptionButton_2 = False And OptionButton_3 = False Then 'Si pas de civilité
    Label_civilite.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_nom = "" Then 'Si pas de nom
    Label_nom.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_prenom = "" Then 'Si pas de prénom
    Label_prenom.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_adresse = "" Then 'Si pas d'adresse
    Label_adresse.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_lieu = "" Then 'Si pas de lieu
    Label_lieu.ForeColor = RGB(255, 0, 0)
ElseIf ComboBox_pays.ListIndex = -1 Then 'Si pas de pays
    Label_pays.ForeColor = RGB(255, 0, 0)
Else

    'Instructions pour insérer le contact ici ...

End If

End Sub
```

EXERCICE PRATIQUE

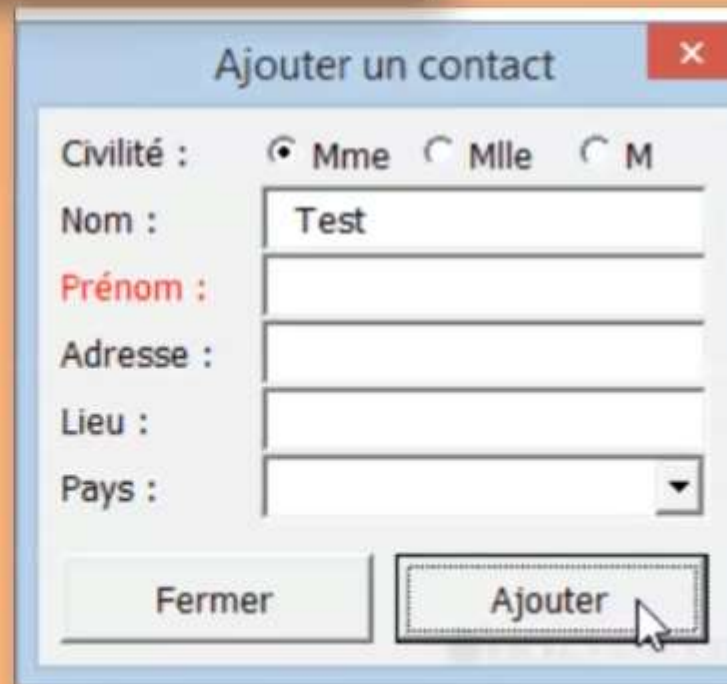
solution

les userform ou formulaires et les controles

EXERCICE PRATIQUE

solution

INSERTION DES DONNÉES



A screenshot of a VBA UserForm titled "Ajouter un contact". The form contains the following controls:

- Civilité :** Three radio buttons labeled "Mme", "Mlle", and "M". The "Mme" button is selected.
- Nom :** A text box containing the text "Test".
- Prénom :** A text box.
- Adresse :** A text box.
- Lieu :** A text box.
- Pays :** A dropdown menu.
- Buttons:** Two buttons at the bottom: "Fermer" (Close) and "Ajouter" (Add). A mouse cursor is pointing at the "Ajouter" button.

les userform ou formulaires et les controles

INSERTION DES DONNÉES

```
Dim ligne As Integer

'Numéro de ligne de la première cellule vide de la colonne 1 en partant du bas de la feuille
ligne = Cells(Rows.Count, 1).End(xlUp).Row + 1

'Choix de civilité
If OptionButton_1 Then
    Cells(ligne, 1) = OptionButton_1.Caption
ElseIf OptionButton_2 Then
    Cells(ligne, 1) = OptionButton_2.Caption
Else
    Cells(ligne, 1) = OptionButton_3.Caption
End If
```

les userform ou formulaires et les controles

INSERTION DES DONNÉES

'Insertion des valeurs sur la feuille

```
Cells(ligne, 2) = TextBox_nom
```

```
Cells(ligne, 3) = TextBox_prenom
```

```
Cells(ligne, 4) = TextBox_adresse
```

```
Cells(ligne, 5) = TextBox_lieu
```

```
Cells(ligne, 6) = ComboBox_pays
```

les userform ou formulaires et les controles

INSERTION DES DONNÉES

```
'Après insertion, réinitialisation du formulaire  
OptionButton_1 = False  
OptionButton_2 = False  
OptionButton_3 = False  
TextBox_nom = ""  
TextBox_prenom = ""  
TextBox_adresse = ""  
TextBox_lieu = ""  
ComboBox_pays.ListIndex = -1
```

les userform ou formulaires et les controles

INSERTION DES DONNÉES

Dans ce code, le numéro de la ligne d'insertion est obtenu grâce à :

```
ligne = Cells(Rows.Count, 1).End(xlUp).Row + 1
```

`Rows.Count` retourne ici le nombre de ligne de la feuille, `Cells(Rows.Count, 1)` correspond donc à la dernière cellule de la colonne 1.

En partant de cette dernière cellule et en recherchant vers le haut (xlUp), Excel va retourner le numéro de ligne de la première cellule non vide de cette colonne (qui sera ici la dernière ligne complétée du tableau). Et puisque l'on ne recherche pas la dernière ligne complétée du tableau mais celle juste au-dessous, il faut encore ajouter `+ 1` à ce numéro.

les userform ou formulaires et les controles

VUE D'ENSEMBLE

Pour terminer, voici le code complet :

```
'Ouverture de l'UserForm
Private Sub UserForm_Initialize()

    Dim i As Integer

    'Boucle pour ajouter les pays dans la liste déroulante
    For i = 1 To 231
        ComboBox_pays.AddItem Sheets("Pays").Cells(i, 1)
    Next

End Sub
```


les userform ou formulaires et les controles

VUE D'ENSEMBLE

'Bouton Ajouter

```
Private Sub CommandButton_ajouter_Click()
```

```
    'Coloration des Labels en noir (&H80000012 = couleur de base de la propriété ForeColor)
```

```
    Label_civilite.ForeColor = &H80000012
```

```
    Label_nom.ForeColor = &H80000012
```

```
    Label_prenom.ForeColor = &H80000012
```

```
    Label_adresse.ForeColor = &H80000012
```

```
    Label_lieu.ForeColor = &H80000012
```

```
    Label_pays.ForeColor = &H80000012
```

les userform ou formulaires et les controles

VUE D'ENSEMBLE

```
'Contrôles des champs
If OptionButton_1 = False And OptionButton_2 = False And OptionButton_3 = False Then 'Si
pas de civilité
    Label_civilite.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_nom = "" Then 'Si pas de nom
    Label_nom.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_prenom = "" Then 'Si pas de prénom
    Label_prenom.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_adresse = "" Then 'Si pas d'adresse
    Label_adresse.ForeColor = RGB(255, 0, 0)
ElseIf TextBox_lieu = "" Then 'Si pas de lieu
    Label_lieu.ForeColor = RGB(255, 0, 0)
ElseIf ComboBox_pays.ListIndex = -1 Then 'Si pas de pays
    Label_pays.ForeColor = RGB(255, 0, 0)
Else
```

les userform ou formulaires et les controles

VUE D'ENSEMBLE

```
Dim ligne As Integer

'Numéro de ligne de la première cellule vide de la colonne 1 en partant du bas de la
feuille
ligne = Cells(Rows.Count, 1).End(xlUp).Row + 1

'Choix de civilité
If OptionButton_1 Then
    Cells(ligne, 1) = OptionButton_1.Caption
ElseIf OptionButton_2 Then
    Cells(ligne, 1) = OptionButton_2.Caption
Else
    Cells(ligne, 1) = OptionButton_3.Caption
End If
```

les userform ou formulaires et les controles

VUE D'ENSEMBLE

```
'Insertion des valeurs sur la feuille
Cells(ligne, 2) = TextBox_nom
Cells(ligne, 3) = TextBox_prenom
Cells(ligne, 4) = TextBox_adresse
Cells(ligne, 5) = TextBox_lieu
Cells(ligne, 6) = ComboBox_pays

'Après insertion, réinitialisation du formulaire
OptionButton_1 = False
OptionButton_2 = False
OptionButton_3 = False
TextBox_nom = ""
TextBox_prenom = ""
TextBox_adresse = ""
TextBox_lieu = ""
ComboBox_pays.ListIndex = -1

End If

End Sub
```

les userform ou formulaires et les controles

VUE D'ENSEMBLE

```
End Sub
```

```
'Bouton Fermer
```

```
Private Sub CommandButton_fermer_Click()
```

```
    Unload Me
```

```
End Sub
```